

Rho: A Foundation for Efficiently Adaptable VLA Models

Rho team¹

Simran Bagaria, Daphne Chen, Dean Fortier, Jianlong Fu,
Michael Harrison, Tess Hellebrekers, Neel Joshi[†], Andrey Kolobov^{♣*}, Dalton Moore,
Galen Mullins[‡], Michael Murray[§], Eduardo Salinas, Reuben Tan

MICROSOFT RESEARCH

Area leads: *Data §Training ‡Engineering †VLM
♣Project lead

General-purpose physical AI models must combine broad visual and linguistic capabilities with precise control across robot embodiments and efficient adaptation to downstream tasks. We introduce Rho, a family of open-weights models for bimanual robotic manipulation trained using a recipe of *regularized progressive adaptation*. This recipe first physically grounds a Phi-based VLM backbone, then couples it to a flow-matching action expert through multi-embodiment pretraining, subjects it to embodiment-targeted horizontal midtraining on a combination of visual-question-answering and robot data, and finally specializes the resulting model via vertical task adaptation. We systematically ablate Rho’s action-expert architecture and the training recipe. Our experiments show that embodiment midtraining gives a particularly significant boost to the effectiveness of downstream adaptation, enabling Rho flavors for dual-arm YAM Box, FR3 Duo, and UR AI Trainer robots to match and outperform open-weight VLAs trained on significantly more data, achieving the strongest overall performance. We further demonstrate the Rho model family’s built-in capacity for online adaptation: an internal latent policy learns from corrective feedback to select observation-conditioned initial conditions for the frozen flow-matching action expert. With as few as 15 corrected episodes, adapting this lightweight module enables Rho to handle task situations at the fringe of its offline finetuning distribution. Together, these results position Rho as both a strong general-purpose robotic manipulation model and a practical foundation for adaptation. We release the base Rho model and midtrained checkpoints for YAM Box, UR AI Trainer, and FR3 Duo to facilitate Rho’s downstream specialization.

Project page: <https://microsoft.github.io/rrobotics>

Code: <https://github.com/microsoft/rrobotics>

Models and data: <https://huggingface.co/collections/microsoft/rho>

Date: September 2026

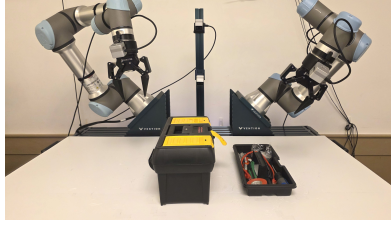
1 Introduction

Physical AI implemented as vision-language-action (VLAs) and world-action models (WAMs) offers a path toward general-purpose robot manipulation by combining the semantic and perceptual capabilities of pretrained vision-language backbones (VLMs) with action generation [3, 13, 19, 20, 22, 43, 72]. Recent models of this type have been exposed to increasingly broad task coverage and expressive continuous control during training. However, zero-shot success rates of existing VLA and WAMs on unseen manipulation tasks and in new environments remain low. *Adaptation* is a crucial and complex step between a pretrained robot foundation model and a version of it that works in practice.

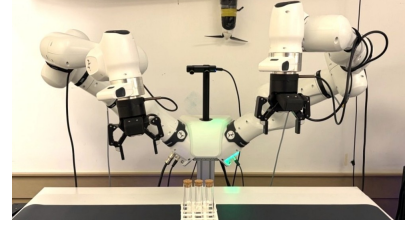
¹Authors are listed alphabetically. See Appendix H for a description of each author’s contributions.



I²RT YAM Box



Universal Robots AI Trainer



Franka FR3 Duo

Figure 1 Rho’s physical dual-arm robot embodiments. YAM Box is common in research labs, FR3 Duo’s usage straddles research and industry, and UR AI Trainer is an emerging standard setup for industrial bimanual manipulation tasks. Midtrained checkpoints for all three are available at <https://huggingface.co/collections/microsoft/rho>

This report introduces Rho, an open-weights family of 5B-parameter physical AI models for bimanual robotic manipulation explicitly designed for data-efficient adaptation to new environments and tasks. Rho’s central premise is that, in order to quickly learn a new task, a physical AI model needs to master vision, language, and control of the specific robot that is expected to perform this task *before* task adaptation begins. Pretraining is insufficient for this: it typically exposes the model to a broad but superficial mixture of tasks, environments, and embodiments, without focusing on sensing, kinematics, action space, and data conventions of any robot in particular. Teaching a pretrained model to complete an unseen task, then, involves familiarizing it with the control of the target robot platform, which makes the adaptation process data-intensive. Rho’s training recipe separates adaptation to a robot embodiment from adaptation to a task, absorbing a large part of the task adaptation burden into the Rho models we present. The Rho family derives from a single foundation, Rho-base, and consists of several VLAs that constitute reusable starting points for adapting to many downstream tasks much more data-efficiently than a pretrained-only model would, as our empirical evaluation demonstrates. We also show that after offline task specialization, these VLAs can continue improving from interaction through an internal latent policy, covering blind spots left by typically small finetuning datasets.

We release the Rho foundation Rho-base along with its variants for three robot platforms spanning the range of de-facto standards for bimanual manipulation research labs and the industry – I²RT YAM Box, Universal Robots AI Trainer, and Franka FR3 Duo (Figure 1). Each of these variants is built via a recipe we call *regularized progressive adaptation*, where each stage adds a form of physical and control training while using physical-grounding VQA data throughout broad action training to help preserve useful visual and linguistic capabilities. As a starting point, Rho uses a backbone distilled from Microsoft’s Phi-family model [2]. Rho’s training recipe adapts this backbone for physical-world comprehension, couples it to a flow-matching action expert that generates continuous action chunks, and pretrains it in the end-effector-pose action space on a multi-task multi-embodiment data mixture that is dominated by demonstrations from dual-arm robot platforms but also includes robotics-relevant Web VQA data. This produces the foundation model Rho-base and is followed by Rho-base’s multi-task single-embodiment midtraining. The embodiment-midtraining stage is key: during it, Rho-base focuses on learning to control a target robot across a broad set of tasks, motions, and environments, giving rise to Rho family’s embodiment-specific VLAs *Rho-YAM-Box*, *Rho-UR-AI-Trainer*, and *Rho-FR3-Duo*. Vertical task adaptation comes after horizontal midtraining and finetunes these VLAs to a target task. An embodiment checkpoint can first be specialized offline using a comparatively small set of task demonstrations. For continued adaptation after deployment, each of Rho-family VLA’s flow module includes an optional internal latent policy that maps the current observation to the flow’s initial condition, replacing random sampling while leaving the action generator fixed. In our online experiments, only this lightweight internal module learns from corrective interaction,

preserving the broader capabilities acquired earlier.

Our experiments assess whether this combination of reusable embodiment checkpoints and online adaptation provides a practical route from a generalist VLA to reliable task execution, both in simulation and on physical robots. We systematically study the architecture of Rho’s action expert and the composition and scheduling of its pretraining mixture. Across the settings and open-weights baselines evaluated in this report, Rho achieves the strongest overall performance.

The main contributions of this report are:

- We introduce Rho, an open-weights, Phi-based physical AI model with a flow-matching action expert, and provide extensive ablations of the action-expert architecture.
- We publish the base Rho checkpoint and its 3 embodiment-midtrained variants Rho-YAM-Box, Rho-UR-AI-Trainer, and Rho-FR3-Duo, for researchers and practitioners to adapt to their own scenarios.
- We systematically evaluate Rho’s multi-stage training recipe that combines physical grounding with heterogeneous, multi-embodiment robot data and per-task adaptation. Our simulation studies are performed in RoboTwin [7], LIBERO [33], and RoboEval [58], and our physical experiments are conducted on FR3 Duo, UR AI Trainer, and YAM Box.
- We empirically analyze the effects of midtraining on downstream adaptation and show that it halves the amount of task finetuning data needed to achieve a given success rate threshold, compared to finetuning pretrained Rho without midtraining. We also demonstrate that Rho can continue improving after offline task specialization by adapting its internal latent policy from corrective interaction.

Together, these results establish Rho as a strong open-weights VLA and, more broadly, examine how architecture, pretraining, horizontal midtraining, and vertical task adaptation interact in a practical robot-learning pipeline.

2 Physically grounded Phi-family backbone

2.1 Backbone architecture

Rho begins from a vision-language model developed within the Microsoft Phi model family. The Phi family emphasizes capability relative to model size and compute through careful architecture, data curation, and staged training [2]. This emphasis is well matched to robot control, where visual-language inference is part of an interactive control loop and must share a deployment budget with action generation.

We refer to the vision-language backbone used by Rho as *Phi-Phy*. It is a compact, 4.68-billion-parameter model with a Phi-family causal language decoder and a SigLIP 2 NaFlex vision encoder [55]. Following the mid-fusion design studied in Phi-4-reasoning-vision-15B [2], it has a two-layer cross-modal projector maps visual features into the language embedding space. The projected visual “soft tokens” are interleaved with the text-token embeddings and processed by the language decoder. This design gives the decoder joint access to visual and linguistic context while retaining a modular visual encoder and language model.

Component	Specification
Full vision-language model	4.68B parameters
Language decoder	32 layers; width 3,072; MLP width 8,192
Decoder attention	32 query heads; 32 key/value heads
Vocabulary and positions	100,352 tokens; 16,384 configured positions
Vision encoder	SigLIP 2 SO400M NaFlex; 428M parameters
Image representation	16×16 patches; 256–3,600 visual tokens
Cross-modal projector	MLP: $1,152 \rightarrow 3,072 \rightarrow 3,072$, with GELU

Table 1 Architecture of the Phi-Phy vision-language backbone used to initialize Rho.

The NaFlex encoder preserves image aspect ratio and varies the number of visual tokens with image size, subject to minimum and maximum patch budgets. Phi-Phy takes features from the penultimate vision-encoder layer, removes padded visual positions, and maps the remaining features through the cross-modal projector. The resulting sequence can contain one or more images interleaved with language. For Rho pretraining, camera views are resized with padding to 256×256 before this encoding step, giving a predictable visual-token budget for multi-camera control.

2.2 Physical grounding data

Name	Description	Example
Attribute Similarity	Find the odd-one-out by font, color, or layout.	Which screenshot uses a different font?
Component Counting	Count a UI component across screenshots.	How many screenshots contain a card?
Object Counting	Count objects across natural images.	How many boats in all images?
Chart Type Matching	Find the odd-one-out by chart type.	Which chart is not a grouped bar chart?
Cross-Chart Reasoning	Combine evidence from related charts.	What patterns emerge across the charts?
Chart Difference Spotting	Identify chart changes and their effects.	What changed; how were profits affected?
UI Difference Spotting	Find components that were added, removed, or restyled.	What changed between these screenshots?
Multi-Image UI Grounding	Locate and compare named UI elements.	Mark the “Support” link in each image.
PIL-Style Transform Detection	Identify PIL-style transforms between paired images.	What image transform occurred?
Multilingual OCR Retrieval	Localize text across multilingual documents.	Which images contain “bonjour”; where?

Table 2 Multi-image data categories and example VQA questions.

General-purpose vision-language posttraining mixture of Phi-4-reasoning-vision-15B provides broad semantic and perceptual knowledge but does not directly emphasize the physical relationships most relevant to robot control, with the exception of PixMo collection [10]. For Phi-Phy, we use additional data that is a mixture of robotics-relevant visual question answering and grounding

data. The objective is to strengthen physical perception and reasoning before the model is asked to learn continuous actions from the comparatively smaller robot-data distribution. The data additions came in three categories:

- High-quality publicly available robotics-relevant data: MGrounding [30], RefSpatial [69], XVR [21], and the object-reference and spatial-reference subsets of RoboPoint [65].
- Multi-image grounding, pointing, OCR, and counting data, generated by reprocessing single-image VQA data in robotics-relevant ways.
- Synthetic multi-image difference-spotting and odd-one-out VQA, created using synthetic-chart and synthetic-UI environments; in particular, with ground truth VQA.

Detailed descriptions of the latter two categories can be found in Table 2. Out of 100M SFT records in the final data mix, roughly 6M fall in the above three categories, with an additional 30M of general caption, perception, object-existence, and UI-heavy grounding, pointing, and relational data.

3 Flow-matching action architecture

Rho converts the physically grounded Phi-family backbone from Section 2 into a vision-language-action model by coupling it to a continuous action expert. At each control step, the model conditions on visual observations, a language instruction, and robot state, and generates a chunk of future actions with a flow-matching objective. This section defines the model interfaces and the architectural choices evaluated later in Section 7.1.

3.1 Observation and action representations

The observation sequence combines one or more camera views with the task instruction and proprioceptive state. Each camera view occupies an image placeholder in the Phi-Phy chat sequence, so visual tokens from multiple cameras are interleaved with the language instruction before the decoder processes them. Proprioceptive state is kept outside this sequence and mapped to the action-expert width by a learned linear projection.

Rho represents robot state and actions as continuous vectors with up to 32 dimensions, without requiring a particular control parameterization. The same interface can support joint-space or end-effector control, delta or absolute targets, and single- or multi-arm embodiments. The 32-dimensional ceiling leaves headroom for higher-dimensional joint-space and auxiliary controls. Dataset adapters pad smaller vectors to the common width and supply masks so that unused state dimensions, action dimensions, and chunk positions do not contribute to the loss.

The model predicts chunks of future actions rather than individual actions [68], which can reduce compounding error and provide ensemble-like generalization across temporal offsets [27]. The prediction horizon and control frequency are configurable properties of a training recipe rather than fixed architectural constants. The execution horizon is likewise a deployment-time choice: a controller may execute the full prediction or replan after a shorter prefix. Rho can also be combined with real-time chunking (RTC) [4] to update chunks asynchronously while maintaining smooth transitions between successive predictions.

3.2 Action expert

The action expert is trained to transform Gaussian noise into continuous robot action chunks conditioned on the backbone representation. Given a ground-truth action chunk a , noise ϵ , and time t , we construct $x_t = t\epsilon + (1 - t)a$ and train the model to predict the velocity $\epsilon - a$ with a mean-squared flow-matching loss. At inference, the learned field is integrated from noise at $t = 1$ toward an action chunk at $t = 0$; the default configuration uses 10 explicit Euler steps.

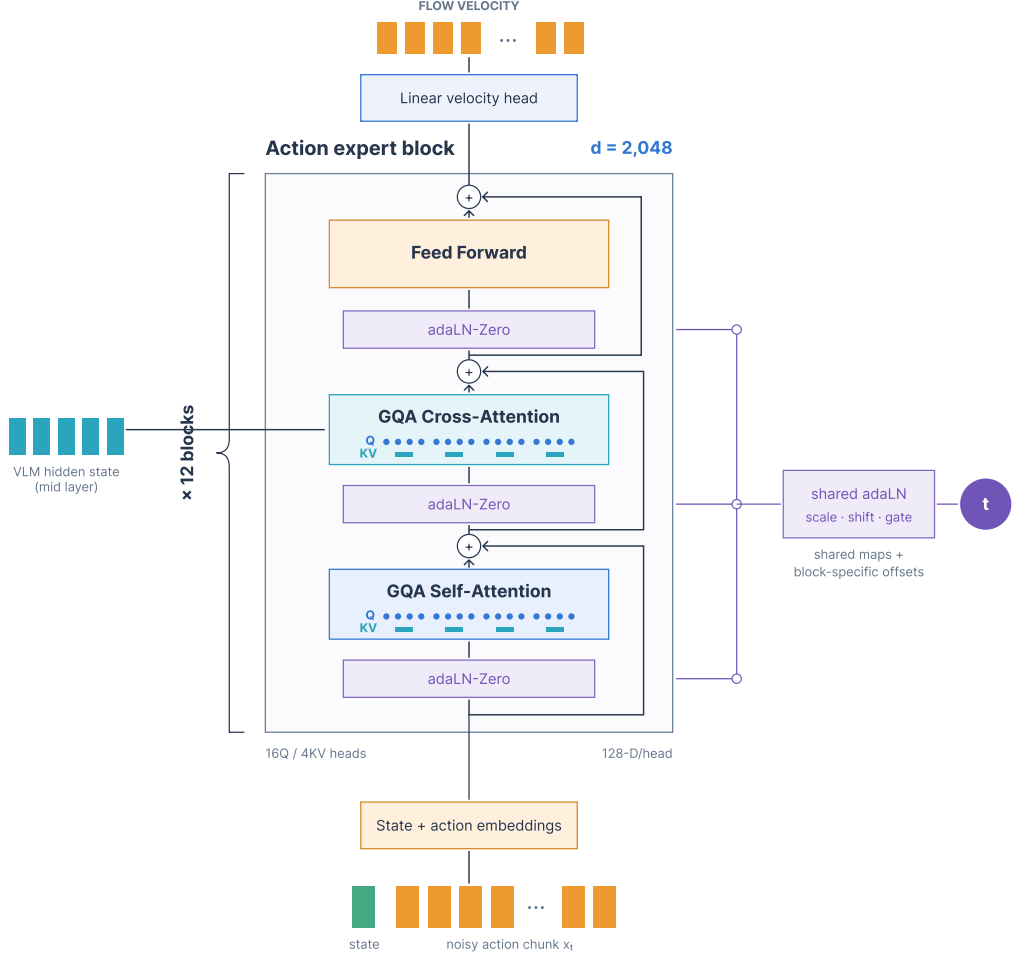


Figure 2 The Rho flow-matching action expert. State and noisy-action tokens pass through 12 wide transformer blocks with grouped-query self-attention, cross-attention to the VLM context, and feed-forward updates. The flow timestep conditions the sublayers through shared adaLN-Zero maps with block-specific offsets.

Robot state and noisy-action tokens are processed by a transformer action expert. Each block applies action-stream self-attention, cross-attention to the Phi-Phy context, and a feed-forward update. Figure 2 and table 3 illustrate the block structure and summarize the complete configuration.

The flow timestep conditions the expert through DiT-style adaLN-Zero, which produces timestep-dependent scale, shift, and residual-gating signals for the transformer updates [46]. Rather than learning a separate modulation network in every block, Rho uses the parameter-efficient adaLN-single design: the base modulation maps are shared across blocks and each block learns a small zero-initialized offset [6, 48]. With 12 blocks, shared modulation, and grouped-query attention, the resulting action expert contains 542 million parameters. These choices preserve the 2,048-dimensional action representation and 128-dimensional attention heads while reducing capacity along dimensions for which our ablations show little benefit.

The selected shape balances per-head resolution with relational parallelism. The wide action stream gives the flow-matching expert capacity to represent high-fidelity conditional velocity fields, while sixteen 128-dimensional heads can retain fine geometric detail and model many relationships among observations, robot state, and actions in parallel. We expect this balance to matter most for contact-rich and precision manipulation, where successful action chunks occupy a narrow region of action space. The task-level RoboEval results in Section 7.1 support this interpretation.

Component	Specification
Action expert	542M parameters
Transformer	12 blocks; width 2,048; MLP width 4,096
Attention	16 query heads; 4 key/value heads; head dimension 128
Block structure	Action self-attention; cross-attention to VLM context; feed-forward network
Flow conditioning	Shared adaLN-single with block-specific offsets
Flow target	Linear noise–action interpolation; velocity-prediction MSE
State/action interface	Continuous vectors with up to 32 dimensions; padding and loss masks
Inference solver	10 explicit Euler steps
Prediction horizon	Configurable action-chunk length
Backbone context	Decoder block 14; learned 3,072 $\rightarrow$ 2,048 projection

Table 3 Configuration of the Rho flow-matching action expert.

3.3 Backbone–expert interface

The interface between the VLM and action expert determines how visual and linguistic information reaches the continuous controller and how action-learning gradients affect the pretrained backbone. Rho reads the joint image–language sequence from an intermediate Phi-Phy decoder representation. A learned linear map projects this 3,072-dimensional sequence to the 2,048-dimensional action-expert space. Every action-expert block cross-attends to this same projected sequence, while the action stream contains the projected robot-state token followed by the noisy action tokens. The VLM context is computed once and reused across all flow-integration steps during inference.

During multi-embodiment pretraining, the vision encoder, cross-modal projector, and language-decoder blocks remain trainable at a small learning rate, while the action expert and robot-specific projections use a larger learning rate. The language token-embedding table remains frozen. Vision-language batches regularize the backbone during multi-embodiment pretraining. We evaluate alternative backbone–expert interfaces, trainable parameter groups, and initialization choices in Sections 7.1 and 7.2.

4 Pretraining

Multi-embodiment multi-task pretraining spanning diverse environments and data sources turns the grounded backbone and action expert into a general continuous-control VLA, with the goal of learning transferable action representations before adapting to a particular physical platform.

4.1 Pretraining data mixture

We call our combined pretraining data mixture *Rho-Tomyum* (Figure 3). Its robot-data portion spans more than a dozen single- and dual-arm embodiments and combines publicly available datasets [44] with proprietary physical-robot and UMI-style [9] demonstrations. It also contains a smaller set of simulated trajectories we have generated in NVIDIA Isaac Sim [40] for the UR AI Trainer embodiment using a method similar to OmniReset [62]. We use only a small subset of the *OXE Magic Soup++* collection [3], which we call *OXE Amuse Bouche* and which omits Fractal, Taco Play, DROID, Stanford Hydra, Furniture Bench, Austin Sailor, and BC-Z datasets. The mixture covers diverse manipulation tasks, environments, camera layouts, action spaces, and control frequencies and is dominated by demonstrations from dual-arm robots, as Rho is geared mainly toward bimanual manipulation. We map observations and actions from each robot source into the padded model interface in Section 3 and apply the pretraining action standardization in Section 4.2. We then draw examples using fixed source weights chosen to balance dataset scale and embodiment coverage rather than sampling a raw concatenation uniformly.

The vision-language portion of Rho-Tomyum consists of RoboPoint [65] and RefSpatial [69],

sampled with equal probability. RoboPoint supplies spatial-affordance pointing examples together with object-reference and general visual-question-answering data. From RefSpatial, we use its single-image 2D and 3D spatial-choice and reasoning examples, including vacant-space pointing and visual-choice tasks; we omit multi-view examples and use only RGB inputs.

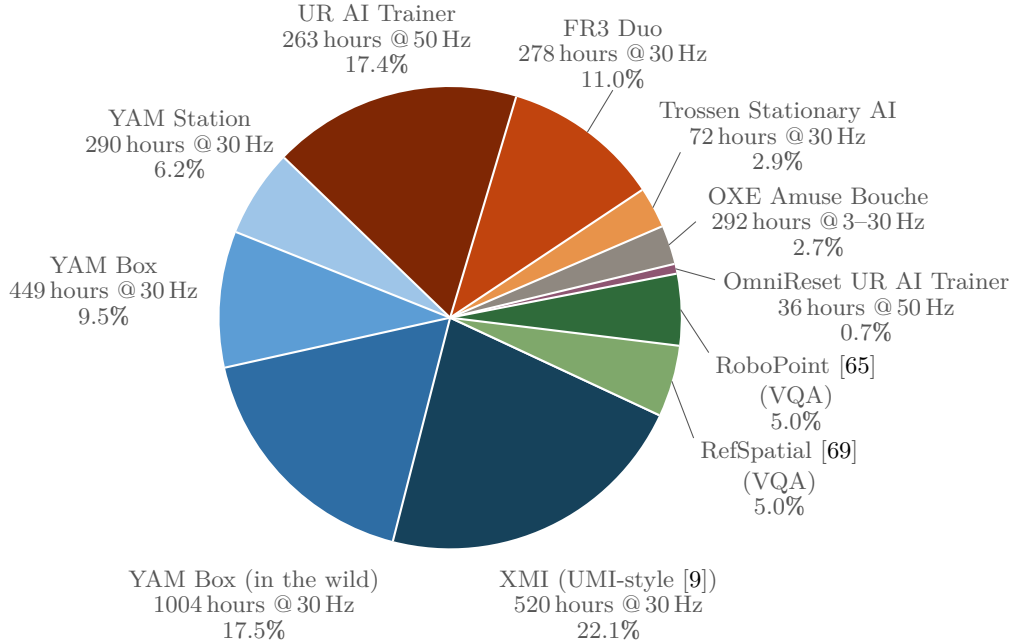


Figure 3 Rho-Tomyum mixture. Its 90% is composed of demonstration trajectories from physical and simulated robots as well as UMI-style demonstrations collected with standard YAM grippers, and the remaining 10% are vision-language data. All the robot/UMI-style data is bimanual, with the exception of a small subset of Open X-Embodiment Magic Soup [3] we call *OXE Amuse Bouche*. The robot data includes trajectories from policies controlling a UR AI Trainer simulated in NVIDIA Isaac Sim, trained using RL and the OmniReset framework [62].

4.2 Robot data standardization

Although Rho can use different continuous control parameterizations within its 32-dimensional interface, we standardize the heterogeneous Rho-Tomyum robot datasets to a common chunk-relative delta end-effector (EEF) representation for pretraining. For each arm and future chunk position k , the target contains a three-dimensional translation delta, a relative orientation in the continuous 6D rotation representation [70], and an absolute gripper position. Both pose deltas are measured relative to the end-effector pose in the current observation at the start of the chunk, rather than relative to the preceding action in the chunk. This gives 10 action dimensions per arm; multi-arm actions are concatenated and then padded to width 32.

For each dataset, we construct a target spanning approximately one second at its native control rate and pad it to at most 50 steps. We normalize actions separately for every source dataset, action dimension, and chunk position: the 1st and 99th percentiles map to -1 and 1 , respectively, and values outside this range are clipped to $[-1.5, 1.5]$. This per-position normalization accounts for the fact that chunk-relative displacement magnitudes generally grow with temporal offset. Current robot state uses per-dataset, per-dimension 1st- and 99th-percentile normalization. Masks remove padded time steps and inactive state and action dimensions from the loss. Our reported embodiment-midtraining and task-adaptation experiments reuse this convention unless otherwise noted, but it is not required by the architecture.

The YAM Box, UR AI Trainer, and FR3 Duo datasets consist of long-horizon tasks requiring high-dexterity, where the errors in task demonstrations are virtually unavoidable. In our raw data, the demonstration segments containing erroneous motions were labeled as such, and we

removed them: the dataset sizes provided in Figure 3 correspond to the clean, processed data with the errors omitted. We note that practices on the inclusion of demonstration errors differ. E.g., $\pi_{0.5}$ [19] uses them in pretraining but eliminates them during posttraining, and $\pi_{0.7}$ [20] includes them while labeling them as errors.

4.3 Pretraining procedure

Training initializes the vision-language backbone from Phi-Phy and randomly initializes the state and action projections and the 12-block action expert described in Section 3. We jointly optimize the vision encoder, cross-modal projector, language decoder, and action modules, while keeping the token-embedding table frozen. Robot examples contain one current observation, a task instruction, proprioceptive state, and up to three RGB camera views; images are resized with padding to 256×256 . Throughout pretraining, embodiment midtraining, and offline task adaptation, we apply color jitter to both scene- and wrist-camera images. The scene view additionally receives mild random in-plane rotation and random cropping, while wrist views receive photometric augmentation only. Robot trajectories use the standardized representation in Section 4.2.

We train with AdamW in bfloat16 using separate peak learning rates of 1×10^{-6} for the pretrained vision-language modules and 1×10^{-4} for the newly initialized action modules. A warmup–stable–decay schedule linearly warms up for 1,000 updates, holds the peak rates, and cosine-decays both rates by a factor of ten over the final 45,000 updates. The full pretraining run uses 48 NVIDIA B200 GPUs. Robot updates use 64 examples per GPU, giving a global robot batch size of 3,072; the longer vision-language updates use 8 examples per GPU, or 384 globally. Table 16 summarizes the recipe. We separately study the effects of component initialization, trainable parameter groups, training schedule, and data-mixture composition in Section 7.2.

4.4 Vision-language co-training

During multi-embodiment pretraining, each optimization update uses either a robot batch or a vision-language batch. With probability 0.9, we sample a robot batch and optimize the flow-matching objective through the backbone and action expert. With probability 0.1, we sample RoboPoint or RefSpatial with equal probability and present the backbone with RGB image inputs, a text prompt, and a target answer. These updates bypass the robot-state and action inputs and train the backbone with autoregressive cross-entropy on visual-question-answering or pointing targets; the action expert is not used for this loss.

We multiply the vision-language cross-entropy loss by 0.02 before backpropagation, while leaving the robot flow-matching loss unscaled. The 9:1 ratio and the 0.02 coefficient therefore control different quantities: the former determines how often each type of update occurs, and the latter controls the gradient scale of a vision-language update. This co-training keeps physically relevant spatial and visual-language supervision in the optimization mixture while the backbone adapts to continuous control. Section 7.2 compares this recipe with otherwise matched robot-only pretraining.

5 Embodiment midtraining

Multi-embodiment pretraining provides a generally-initialized foundation model, but deployment on a physical platform still requires learning its observation layout, kinematics, action space, control conventions, and characteristic workspace. Rho separates this embodiment adaptation from task adaptation. The embodiment midtraining stage adapts the Rho foundation to a target robot type horizontally across many tasks and motions, producing a checkpoint that can be reused for subsequent data-efficient vertical adaptation to individual downstream tasks.

5.1 Embodiment adaptation

Each embodiment checkpoint is initialized from pretrained Rho and trained for 2 epochs over a broad multitask dataset from the target platform. We refer to the resulting checkpoints as Rho-FR3-Duo, Rho-UR-AI-Trainer, and Rho-YAM-Box. Midtraining uses the action standardization in Section 4.2 and preserves the pretraining co-training recipe: robot and vision-language updates are sampled in a 9:1 ratio, and the vision-language loss is weighted by 0.02. Thus, the model adapts to a platform’s observations, kinematics, and control conventions while continuing to receive the spatial and visual-language supervision used during pretraining. Figure 4 shows the composition of each checkpoint’s midtraining mixture. Experiments studying the effect of embodiment midtraining are presented in Section 7.3; the adaptability of Rho-FR3-Duo, Rho-UR-AI-Trainer, and Rho-YAM-Box to physical-robot tasks is assessed in Sections 7.5.1, 7.5.2, and 7.5.3.

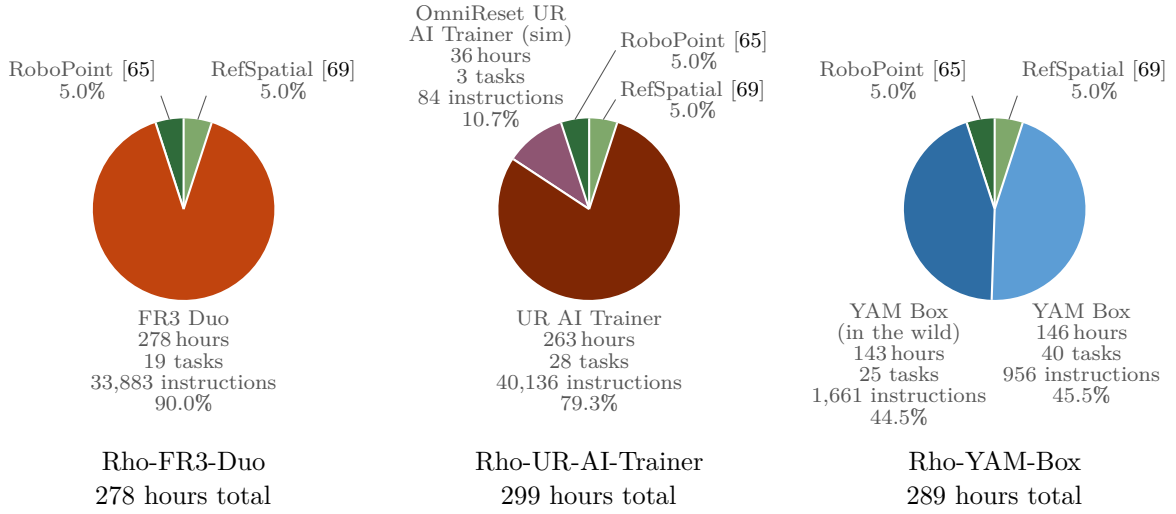


Figure 4 Midtraining mixtures of the embodiment-specific Rho variants. The count of distinct language instructions includes the number of both task and subtask descriptions.

We emphasize that the goal of embodiment adaptation is to familiarize Rho with the controls and visuals from the embodiment’s cameras across a diverse set of tasks rather than to have it master any task in particular. Note that the average number of hours per task in the midtraining datasets (e.g., $278/19 \sim 9$ for UR AI Trainer) is no more than 15 – this is generally insufficient to achieve 90+% success rates on high-precision long-horizon tasks but is helpful for representing a broad distribution of motions in several hundreds of hours of data.

5.2 YAM Box

Rho-YAM-Box is a version of Rho for the dual-arm YAM Box platform trained using a roughly 300-hour subset of the available data, sampled across 65 task families. The mixture is divided approximately evenly between demonstrations from controlled laboratory and demonstrations collected in everyday environments (146 and 143 hours, respectively, sampled from the pretraining YAM Box datasets in Figure 3 uniformly at the episode level). Its tasks range from battery loading, cable winding, terminal-block and faucet installation, and tool-head replacement to folding clothing, cleaning office equipment, cutting with scissors, assembling Lego pieces, organizing household objects, and inserting earbuds into a charging case. This combination exposes the model to precise insertion and assembly, articulated and deformable objects, tool use, and long-horizon bimanual manipulation under substantially more varied visual conditions than the laboratory data alone.

5.3 UR AI Trainer

Rho-UR-AI-Trainer is midtrained on approximately 260 hours of physical demonstrations spanning 28 task families on the dual-arm platform, augmented with approximately 36 hours of successful simulated trajectories covering 3 tasks, collected in NVIDIA Isaac Sim [40]. The physical data includes tool and electronics packing, object sorting, cable routing and connection, storage-device insertion, garment handling, and protective wrapping. To increase repetition on precise contact and placement behaviors, we complement real-world data with simulated rollouts collected on the exact same robotic embodiment and camera setup. This simulated dataset adds three structured manipulation families: cube stacking, left- and right-arm placement, and left- and right-arm insertion. The placement tasks span high object diversity: they involve 10 unique everyday objects in over 80 distinct source-and-target combinations. To improve robustness and generalization, all simulated data incorporates domain randomization across object color, camera positions, and focal lengths.

5.4 FR3 Duo

Rho-FR3-Duo adapts Rho to a dual-arm Franka platform using about 280 hours of physical demonstrations spanning 19 task families. The collection covers bimanual and contact-rich behaviors at different time scales, including surface cleaning, folding and stacking towels, sorting utensils and other objects, pouring between containers, setting tables, packing and unpacking, and inserting cables into computer ports. These tasks require the checkpoint to learn both broad workspace motions and precise coordination between the two arms.

6 Vertical task adaptation

Embodiment midtraining produces a reusable platform-specific checkpoint, but reliable deployment still requires vertical specialization to the target task. We study two complementary forms of vertical adaptation: offline supervised learning from task demonstrations and online learning from interaction. The online methods operate in the model’s latent action space and are evaluated as deployment-time extensions of Rho.

6.1 Offline supervised adaptation

Offline adaptation fine-tunes either pretrained Rho or an embodiment-midtrained checkpoint on demonstrations of the target tasks. We retain the flow-matching objective, action representation, normalization, and image augmentation used during pretraining, and optimize the task policy with AdamW. Unlike pretraining and embodiment midtraining, this stage uses only target-task robot demonstrations; we do not interleave vision-language batches. All trainable parameters, including the backbone and action expert, use a single peak learning rate of 10^{-4} . The optimization budget and schedule are selected for each downstream setting and reported with its evaluation protocol. The controlled experiments in Section 7.3 hold the downstream data and optimization budget fixed while varying the initialization, isolating the contribution of embodiment midtraining.

6.2 Online adaptation in latent action space

Offline task adaptation is limited by the states represented in its demonstration dataset: once deployed, a policy can encounter configurations in which its learned behavior is unreliable. Rho addresses these failures through an optional internal latent policy. Let $G_\theta(o, z)$ denote its flow-matching action generator, which maps an observation o and latent initial condition z to an action chunk. Standard inference draws z from a Gaussian distribution. For online adaptation, Rho instead samples from a learned, observation-conditioned latent policy $\pi_\phi(o)$,

$$a = G_\theta(o, \pi_\phi(o)). \quad (1)$$

The vision-language backbone and action generator G_θ remain frozen; only the lightweight latent policy is updated. The latent policy is a submodule of Rho, stored and served in the same checkpoint rather than applied as an external policy wrapper.

We train this latent policy from corrective interaction. For an observation o and corrected action chunk $a^\star$, we invert the frozen flow to recover a latent target $z^\star \approx G_\theta^{-1}(o, a^\star)$ and regress $\pi_\phi(o)$ toward that target. The corrected chunk may combine autonomous actions with actions supplied after a human or scripted-expert takeover; in either case, inversion translates the behavior that should have been executed into supervision in Rho’s native latent action space. Online learning therefore changes which initial conditions Rho supplies to its pretrained flow, while every resulting action chunk is still generated by the unchanged offline-trained backbone and action expert. This provides a compact adaptation interface and retains the behavioral structure learned from offline data. Related work also uses generative-policy latents for corrective supervision or reward-driven adaptation [38, 56]; our experiments study the corrective-supervision setting.

Section 7.6 evaluates this procedure in simulation and on FR3 Duo. Flow inversion, fixed-point refinement, the internal latent-policy architecture, correction interfaces, and optimization settings are specified in Appendix F.

7 Empirical evaluation

Rho’s empirical evaluation analyzes questions:

- (i) How data-efficient is Rho’s training recipe for adapting to new tasks, compared to the conventional pretraining/offline supervised finetuning approach?
- (ii) How performant are task policies derived from Rho’s midtrained checkpoints using standard supervised finetuning?

By answering them, we show that midtrained Rho variants can be expected to yield reduction in the amount of downstream adaptation data to achieve a given task performance level and, viewed differently, achieve significantly higher success rates when trained on the same data as strong existing VLAs $\pi_{0.5}$, GR00T-N1.7, and MolmoAct2.

We organize the evaluation around the main stages of Rho’s training and adaptation pipeline. We first isolate architectural and pretraining choices in Sections 7.1-7.2, showing that the shape of Rho’s action expert, the inclusion of VQA data into the pretraining mixture, and the Rho-Tomyum pretraining mixture as a whole make a major difference for downstream task performance (Tables 4-5, Figure 5). We then measure the effect of embodiment midtraining in Section 7.3: when compared to pretrained-only Rho finetuned on target tasks, midtrained Rho tends to achieve the same success rates using *half* as much data under the same compute budget (Figure 6). In Sections 7.4 and 7.5 we analyze the performance of Rho and its midtrained versions across several simulated environments and physical robots, showing major improvements in success rates yielded by the Rho family over the strongest open-weights VLAs and noting task properties that influence the size of performance gap between Rho and its baselines (Figures 8 - 11, Table 8). The final experiments, in Section 7.6, study online supervised adaptation of Rho from human feedback and show the data-efficiency of combining it with Section 7.5’s prior offline finetuning from a small amount of demonstrations.

Taken together, our experiments identify the most data-efficient method for adapting Rho: finetuning Rho starting from an embodiment-midtrained checkpoint offline via conventional offline SFT followed by rapid performance improvement through online latent-space adaptation from a few human corrections [38, 56].

7.1 Action-architecture ablations

We evaluate the design choices introduced in Section 3 under a common downstream-adaptation and evaluation protocol. Each model is pretrained for approximately one pass over a 15% subset of the Rho-Tomyum mixture and then adapted with its corresponding action-expert architecture. These controlled pretraining runs use 16 NVIDIA B200 GPUs, with global robot and physical-grounding batch sizes of 1,024 and 128, respectively. Downstream adaptation uses four NVIDIA B200 GPUs with a global batch size of 128. We evaluate all eight RoboEval tasks using 100 rollouts per task in each of three evaluation passes. We first isolate the action-expert shape and then reduce parameters while preserving the dimensions identified by this sweep.

Variant	Width	Heads	Head dim.	Blocks	Success
1,024 / 16 heads / 16 blocks	1,024	16	64	16	0.617
1,024 / 8 heads / 16 blocks	1,024	8	128	16	0.640
1,024 / 8 heads / 32 blocks	1,024	8	128	32	0.642
2,048 / 8 heads / 16 blocks	2,048	8	256	16	0.623
2,048 / 16 heads / 16 blocks	2,048	16	128	16	0.673

Table 4 Action-expert shape sweep on RoboEval after 40k adaptation steps. Success rates are averaged over eight tasks and three evaluation passes.

Table 4 identifies two important dimensions. First, a head dimension of 128 performs best at both widths: at width 1,024, eight 128-dimensional heads outperform sixteen 64-dimensional heads (0.640 versus 0.617), while at width 2,048, sixteen 128-dimensional heads outperform eight 256-dimensional heads (0.673 versus 0.623). Second, width remains important after fixing the head dimension at 128: the 2,048-wide model reaches 0.673, compared with 0.640–0.642 for the 1,024-wide models. Doubling the depth of the 1,024-wide expert from 16 to 32 blocks does not close this gap.

The sweep identifies a useful balance: 128-dimensional heads provide sufficient representational resolution for fine continuous geometry, while the wider 16-head representation allows many spatial, visual, and contact relationships to be modeled in parallel. This balance complements the flow-matching objective: the action expert regresses a conditional velocity field over continuous action chunks, and precision tasks admit only a narrow set of successful trajectories. Resolving how the field should change around this narrow set places greater demands on representational fidelity than a high-tolerance task with many successful actions.

The task-level results are consistent with this interpretation: the gains from the 2,048-wide expert are concentrated on difficult contact and precision tasks, including lifting a flat book from the table, stacking blocks, and placing a single book on a shelf.

Having identified width 2,048 and head dimension 128 as the important shape parameters, we preserve both while reducing capacity elsewhere. Table 5 summarizes this compression sequence.

Variant	Q/KV heads	Blocks	AdaLN	AE params.	Success
Wide baseline	16/16	16	per-block	1.649B	0.673
Shared AdaLN	16/16	16	shared	882M	0.673
Grouped-query	16/4	16	shared	693M	0.672
Rho	16/4	12	shared	542M	0.671

Table 5 Action-expert compression on RoboEval while preserving width 2,048 and head dimension 128. Success rates are averaged over eight tasks after 40k adaptation steps. Each variant uses three evaluation passes.

Sharing the AdaLN modulation maps reduces the action expert from 1.649B to 882M parameters

while matching the baseline performance. Grouped-query attention retains 0.672 success with 693M parameters, and the 12-block configuration retains 0.671 ± 0.008 success with 542M parameters. Together, these reductions preserve the action-expert shape selected above while reducing its parameter count by approximately two thirds.

We also test routing successive action-expert blocks to different Phi-Phy hidden states. After 40k adaptation steps, layerwise cross-attention reaches 0.605 ± 0.008 RoboEval success, compared with 0.673 ± 0.015 when every block attends to the same projected hidden sequence. Rho therefore uses the representation from Phi-Phy decoder block 14 throughout the action expert, yielding higher accuracy with a simpler, more memory-efficient interface.

The same architecture choices remain strong on LIBERO, where the evaluated variants are close to the benchmark ceiling. At 40k adaptation steps, the 1,024-wide baseline reaches 0.973 success, the 2,048-wide expert with shared AdaLN reaches 0.978, and with the final large-scale pretraining checkpoint, the 12-block architecture reaches 0.983. We treat LIBERO as corroborating evidence here: its saturated success rates make it less discriminative than RoboEval for selecting among action-expert variants.

7.2 Pretraining-recipe ablations

We next evaluate how multi-embodiment pretraining affects downstream adaptation. For these controlled experiments, each pretraining run uses approximately one pass over a 15% subset of the robot-data mixture. We then adapt the resulting checkpoints on LIBERO and RoboEval using the same data, optimization budget, and evaluation protocol for every initialization. Results are averaged over three evaluations; LIBERO uses seeded evaluations and reports success separately for each of its four suites, while RoboEval uses complete evaluation passes and averages success over all eight tasks.

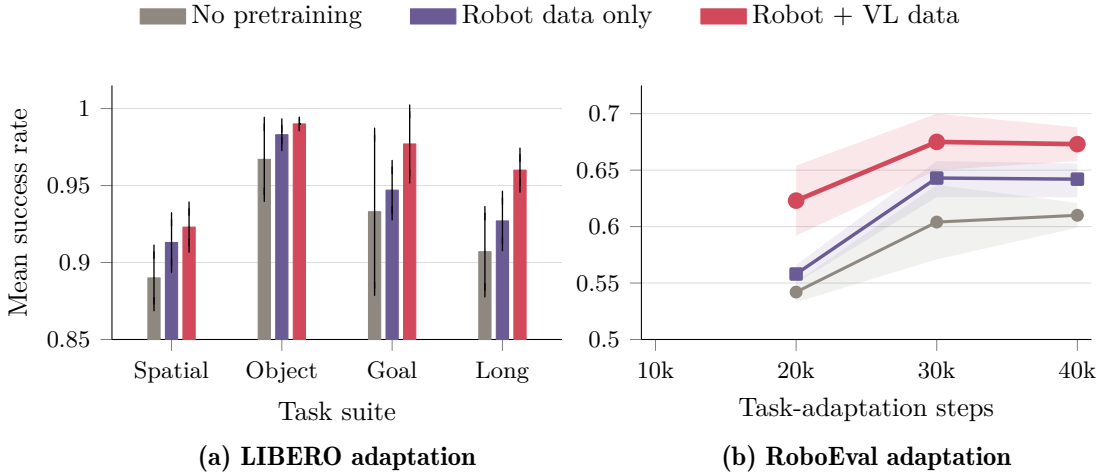


Figure 5 Pretraining on 15% of the dataset mixture improves downstream adaptation. Left: LIBERO suite success after 20k task-adaptation steps. Right: RoboEval success throughout adaptation. The robot+VL condition combines robot trajectories with physical-grounding examples, the robot-data-only condition removes the physical-grounding batches, and the no-pretraining condition begins task adaptation from the grounded backbone with a randomly initialized action expert. Values are means and standard deviations over three evaluations, except the robot+VL RoboEval result at 40k adaptation steps, which pools six passes from two independent evaluations.

Figure 5 shows that the robot+VL pretraining condition improves downstream performance across the LIBERO suites and both adaptation speed and final performance on RoboEval. At 20k task-adaptation steps, robot+VL reaches 0.963 average LIBERO success, compared with 0.943 for robot-data-only pretraining and 0.924 without pretraining. On RoboEval, robot+VL reaches 0.623

after 20k task-adaptation steps and 0.673 after 40k, versus 0.558 and 0.652 for robot-data-only pretraining and 0.542 and 0.610 without pretraining.

The suite-level view in Figure 5 shows that physical-grounding co-training improves all four LIBERO suites at 20k adaptation steps. Relative to robot-only pretraining, success increases from 0.913 to 0.923 on LIBERO-Spatial, 0.983 to 0.990 on LIBERO-Object, 0.947 to 0.977 on LIBERO-Goal, and 0.927 to 0.960 on LIBERO-Long. LIBERO-Object is already near saturation, while the largest gains occur on Goal and Long, where adaptation must connect the instruction and scene to goal-directed, multi-stage behavior. Physical-grounding supervision therefore provides its clearest benefit on tasks with greater semantic and temporal structure.

Having established the benefit of the robot+VL mixture, we next isolate optimization of the action expert. The action modules are randomly initialized and trained at a substantially higher learning rate than the pretrained Phi-Phy backbone, making their learning rate an important control on how aggressively the model acquires continuous-control structure from the heterogeneous robot mixture. We sweep the action-expert learning rate while retaining physical-grounding data and holding the data mixture, backbone learning rate, schedule, and pretraining budget fixed.

Pretraining learning rate	20k adaptation	30k adaptation	40k adaptation
5×10^{-5}	0.601 ± 0.028	0.645 ± 0.005	0.641 ± 0.006
1×10^{-4}	0.623 ± 0.031	0.675 ± 0.035	0.673 ± 0.015
2×10^{-4}	0.586 ± 0.010	0.629 ± 0.031	0.633 ± 0.008
5×10^{-4}	0.548 ± 0.006	0.637 ± 0.021	0.622 ± 0.022

Table 6 Effect of the action-expert learning rate during pretraining on downstream RoboEval adaptation. All rows are pretrained on the same 15% subset of the dataset mixture and include physical-grounding data. The results are mean success and standard deviation over three evaluation passes.

Table 6 selects 10^{-4} : it performs best at every RoboEval evaluation point and reaches 0.673 success after 40k adaptation steps, compared with 0.633 at 2×10^{-4} and 0.622 at 5×10^{-4} . The gap persists throughout downstream adaptation, showing that the pretraining learning rate affects the quality of the transferred initialization rather than only early adaptation speed. We therefore use 10^{-4} for the Rho pretraining recipe.

7.3 Embodiment midtraining experiments

We study two complementary effects of horizontal midtraining. First, in a controlled RoboTwin 2.0 simulation experiment, we measure the effect of robot midtraining on finetuning data efficiency for several held-out tasks, showing that midtraining halves the amount of offline finetuning data needed to achieve a given success rate across the full range of finetuning data regimes. Second, in a physical FR3 Duo experiment we keep the target task data and adaptation budget fixed to measure the effect of the amount of midtraining *compute*, as measured by the number of midtraining epochs, on real-world task-adaptation efficiency.

7.3.1 Controlled analysis of midtraining transfer

We use RoboTwin 2.0 [7] for a controlled test of embodiment midtraining. RoboTwin provides demonstration data for 50 tasks across five simulated robot embodiments. We select the dual-arm UR5e as the target bimanual embodiment and divide its tasks into disjoint horizontal-midtraining and vertical-adaptation subsets. We then compare task adaptation efficiency starting from the base Rho and from a Rho checkpoint midtrained on the data from the midtraining tasks.

For this experiment, 45 tasks are used for embodiment midtraining and 5 are held out for downstream adaptation. For each held-out task, we construct nested demonstration subsets containing 12.5%, 25%, 50%, and 100% of the available data. We combine the corresponding

subsets across tasks to form adaptation datasets $\mathcal{D}_{0.125}$, $\mathcal{D}_{0.25}$, $\mathcal{D}_{0.5}$, and $\mathcal{D}_{1.0}$. Both pretrained and embodiment-midtrained initializations are adapted on each dataset under the same 40k-step optimization budget and evaluated on the same held-out tasks under both the Easy and Hard RoboTwin protocols.

Figure 6 reports the results. Across all finetuning data budgets until performance saturation, midtraining reduces the amount of finetuning data needed for achieving a given success rate by half: e.g., the midtrained Rho adapted on 25% of the finetuning data performs as well as the base Rho adapted on 50% of the finetuning data. For a different perspective, in the low-finetuning-data regime of 12.5%, the midtrained Rho performs 30% better than the finetuned base Rho. These effects hold in Easy and Hard evaluation regimes.

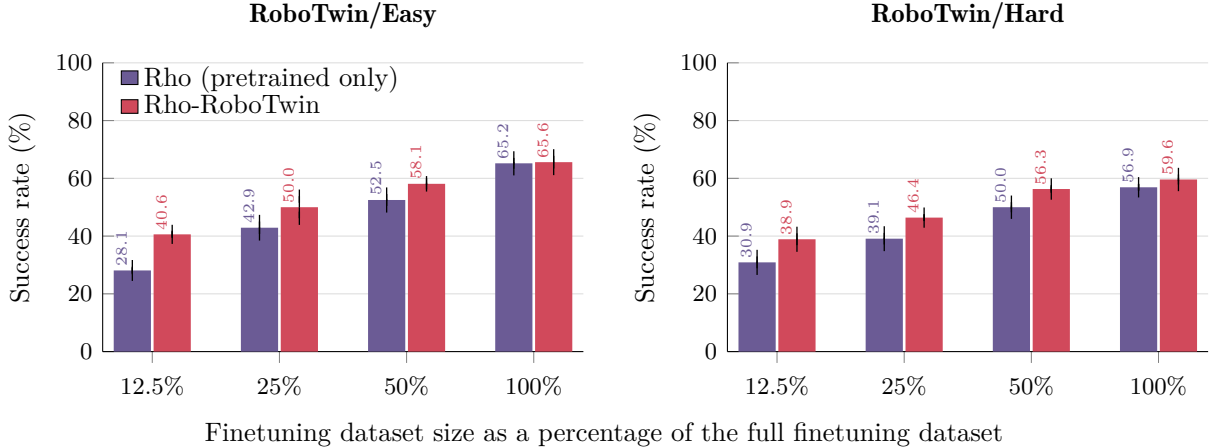


Figure 6 Embodiment midtraining on RoboTwin transfers to held-out tasks across downstream-data budgets. Each checkpoint is adapted for 40k steps and evaluated on the matched five-task protocol with 50 episodes per task. Bars are means over completed seeds (four for Easy, three for Hard) and error bars are sample standard deviations. The benefit of midtraining is largest in the low-data regime and shrinks as the adaptation set grows.

7.3.2 Real-world task-adaptation efficiency

We additionally isolate the effect of physical-robot horizontal midtraining on FR3 Duo. All conditions begin from the same pretrained Rho checkpoint. We either adapt it directly or first midtrain it for one or two epochs on the FR3 Duo mixture, then adapt each initialization for 50k steps on the same bimanual plug-insertion data. As shown in Figure 7, success over 30 evaluation trials increases from 18/30 (60.0%) without horizontal midtraining to 23/30 (76.7%) after one epoch and 26/30 (86.7%) after two epochs.

Qualitatively, much of the improvement appears on initial plug configurations near the edge of the task-training distribution, including rare or unusual positions within the workspace. We hypothesize that broad FR3 Duo midtraining gives the policy a stronger model of the platform’s reachability and motion geometry, helping it plan through workspace regions not represented in the plug-insertion demonstrations.

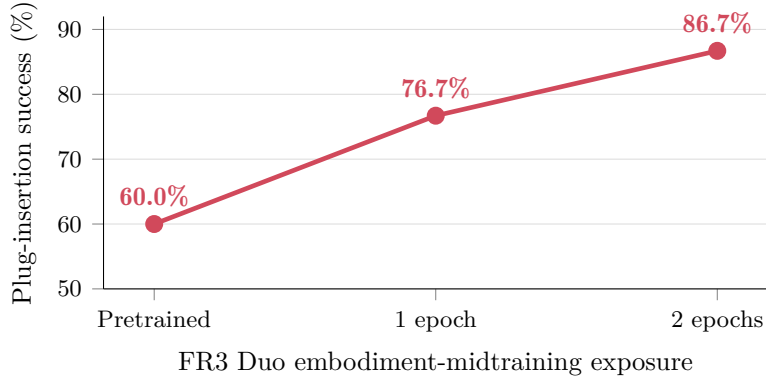


Figure 7 FR3 Duo horizontal midtraining improves subsequent vertical adaptation to bimanual plug insertion. Every condition uses the same 50k-step task-adaptation budget; success reported over 30 trials.

7.4 Rho and existing VLAs: performance comparison in simulation

In this section, We compare the performance of task-adapted Rho-base policies and policies finetuned from the strongest existing open-weight VLAs on LIBERO [33] and RoboEval [58]. For each benchmark, we follow its standard protocol as detailed in Appendix C and compare against baselines trained with matched benchmark data.

On RoboEval, we compare finetuned Rho-base’s performance against finetuned MolmoAct2 point estimates reported in the MolmoAct2 evaluation [13] and the results with our evaluations of GR00T N1.7 and $\pi_{0.5}$. We adapt Rho-base for 40k optimization steps with a batch size of 128 and 8 flow samples per training example; the complete adaptation and evaluation protocol is given in Appendix C. Rho-base reaches 63.7% average success on RoboEval, outperforming MolmoAct2 by 5.0 percentage points. It achieves the highest task-level success on 3 of the 8 tasks, with the largest margin on *Stack Single Book on Shelf*.

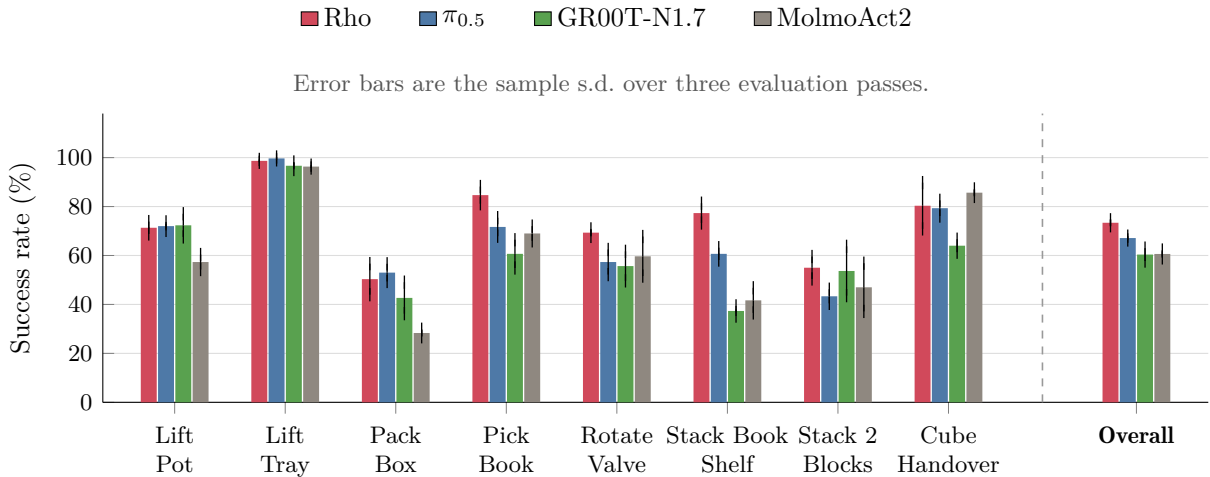


Figure 8 Success rates of finetuned Rho, $\pi_{0.5}$, GR00T-N1.7, and MolmoAct2 on RoboEval tasks. Rho outperforms all other models on *Pick Book*, *Stack Book Shelf*, and *Rotate Valve*. On each of the remaining 5 tasks, Rho matches the performance of the best baseline on that task. Importantly, each task’s best-performing baseline is generally different, so the next-strongest model, $\pi_{0.5}$, matches Rho’s performance on only 4 out of 8 tasks.

Model	Succ. $\uparrow$	TP $\uparrow$	BGVD $\downarrow$	CPL $\downarrow$	ECC $\downarrow$	JPL $\downarrow$	OPL $\downarrow$	SCC $\downarrow$	SC $\downarrow$
GR00T N1.7	0.61	0.80	0.078	1.66	1.25	9.64	5.43	0.095	0.540
$\pi_{0.5}$	0.67	0.82	0.074	1.45	0.99	7.98	4.82	0.038	0.482
MolmoAct2	0.61	0.79	0.072	1.48	1.03	7.97	4.84	0.017	0.465
Rho	0.73	0.85	0.073	1.41	1.02	7.70	4.69	0.035	0.477

Succ.: success rate; TP: task progression; BGVD: bimanual gripper vertical difference (m); CPL: Cartesian path length (m); ECC: environment collision count; JPL: joint path length (rad); OPL: orientation path length (rad); SCC: self-collision count; SC: slip count.

Table 7 Aggregate RoboEval outcome and behavioral metrics after 40k task adaptation steps. Rho outperforms the baselines on 5 out of the 9 metrics, with the next-strongest model across these metrics, MolmoAct2, winning on 3. Every row is the mean of 3 800-episode evaluations across the 8 task variations. *Joint Path length (JPL)* measures the sum of 3D path lengths traced by the two arms. Arrows indicate the preferred direction of each metric, e.g., “ $\downarrow$ ” means that lower is better.

Beyond binary success on RoboEval tasks, Table 7 reports task progression and behavioral metrics for coordination, motion efficiency, and safety. In addition to mean success rate, Rho achieves the highest task progression and the shortest Cartesian, joint, and orientation paths; $\pi_{0.5}$ has the fewest environment collisions, while MolmoAct2 has the lowest bimanual gripper vertical difference, self-collision count, and slip count. We provide the full task-level breakdown in Appendix G.

For LIBERO, we compare finetuned Rho-base against representative open VLA baselines with publicly reported LIBERO results in Table 8. For OpenVLA, π_0 , and MolmoAct-7B-D [] we cite their success rates from the original model reports [3, 22, 28]. The $\pi_{0.5}$ values come from the official OpenPI LIBERO release [47]. The GR00T N1.7 results come from NVIDIA’s official LeRobot evaluation [41]. Rho-base reaches 97.9% average success across LIBERO’s 4 task suites, the strongest aggregate result among the compared models. Rho-base’s margin comes almost entirely from LIBERO-Long, the hardest suite, where it reaches 95.9% and exceeds the next-best model by 2.7 percentage points; on the three shorter-horizon suites it is within 0.5 points of the best reported value.

Model	Spatial	Object	Goal	Long	Average
OpenVLA [22]	84.7	88.4	79.2	53.7	76.5
π_0 [3]	96.8	98.8	95.8	85.2	94.2
MolmoAct-7B-D [28]	87.0	95.4	87.6	77.2	86.6
GR00T N1.7 [41]	95.0	100.0	98.0	93.0	96.5
$\pi_{0.5}$ [47]	98.8	98.2	98.0	92.4	96.9
MolmoAct2 [13]	97.8	100.0	97.8	93.2	97.2
Rho	98.3	99.6	97.6	95.9	97.9

Table 8 Rho-base’s mean performance and published success rates of several baselines on the 4 task suites of LIBERO [33]. Rho-base outperforms the baselines in overall success rate aggregated across all 4 task suites and in success rate on the most challenging suite, *LIBERO-long*. Baseline values are taken from the corresponding model report or official release as cited in column 1; evaluation and adaptation protocols may differ across the reports. For Rho-base, the mean success rate is over 3 seeds. All values are percentages. **Bold** denotes the best result in each column.

7.5 Rho and existing VLAs: performance comparison on physical robots

In the central set of experiments of this report, we evaluate task adaptation of Rho-family models on physical robots, some of the most common stationary dual-arm embodiments across research labs and the industry: FR3 Duo, UR AI Trainer, and YAM Box (Figure 1). Unlike the evaluations in Section 7.4, which focused on adapting Rho-base, the physical-robot experiments examine the

adaptability of Rho-YAM-Box, Rho-UR-AI-Trainer, and Rho-FR3-Duo – the Rho-base derivatives midtrained for the robots we use. On each embodiment, we compare the corresponding Rho variant against $\pi_{0.5}$ and GR00T-N1.7; on YAM Box, we additionally evaluate against MolmoAct2, since this model has significant amounts of data from a dual-arm YAM robot similar to YAM Box in its training mixture (720 hours). Each model is adapted independently on the same task-specific demonstrations before evaluation. The comparisons are carried out according to the parallel A/B evaluation [26], which minimizes the effect of environment drift and the choice of initial configurations of the task environments on the experiment outcome. Robot specifications and full evaluation protocols are provided in Appendices D and E.

The evaluations span advanced stacking and pick-and-place tasks, packing and sorting, insertion, and interaction with deformable as well as articulated objects – all requiring precision during execution. Together, these tasks reflect realistic manipulation challenges and probe long-horizon bimanual coordination, precise control, generalization across environment configurations, and a broad range of manipulation skills.

7.5.1 YAM Box

The YAM Box study considers a high-data finetuning regime: we finetune $\pi_{0.5}$, GR00T-N1.7, MolmoAct2, and Rho-YAM-Box on ~ 2000 demonstrations representing 6 task categories from the BusyBox [14] manipulation benchmark, which we describe here for completeness:

- **Pushing buttons.** The robot must press one of four colored, illuminated buttons with a specified gripper (e.g., “Push the blue button with the left gripper”).
- **Flipping switches.** The robot must flip a specified switch on or off using the specified arm (left or right), as described in the task instruction. Because flipping requires considerable force, one arm must pin the BusyBox in place while the other operates the switch, making the task bimanual.
- **Moving sliders.** The robot must move the top or the bottom slider, as dictated by the task instruction, to a specified position between 1 and 5. The task instruction doesn’t specify with which arm this should be done, so the task involves selecting the correct slider, choosing the arm, and positioning the slider accurately.
- **Rotating the knob.** The robot must turn the knob to a specified position between 1 and 6, which can be made difficult by occlusions.
- **Pulling wires.** The robot must grasp and unplug a wire of a specified color (red, black, blue, or white) from the wire module.
- **Repositioning the BusyBox.** The robot must rotate the entire BusyBox clockwise or counter-clockwise, or slide it left, right, closer, or away. The task variants involving rotations are bimanual.

The full finetuning dataset for YAM Box that we used in this experiment is available at <https://huggingface.co/datasets/microsoft/BusyBox>.

We pool the data from all tasks and finetune Rho-YAM-Box on the entire dataset for 50k steps with a global batch size of 128. The baselines were finetuned with their recommended finetuning settings in their respective repositories or reports OpenPi [18], GR00T N1.7 [39], and MolmoAct2 [13]. We evaluate each finetuned model’s success rate over 60 rollouts, 10 rollouts per task category. Each of a task’s 10 rollouts uses a different instruction (e.g., “*Flip the bottom switch on with the left gripper*” vs. “*Flip the bottom switch off with the right gripper*”) and a

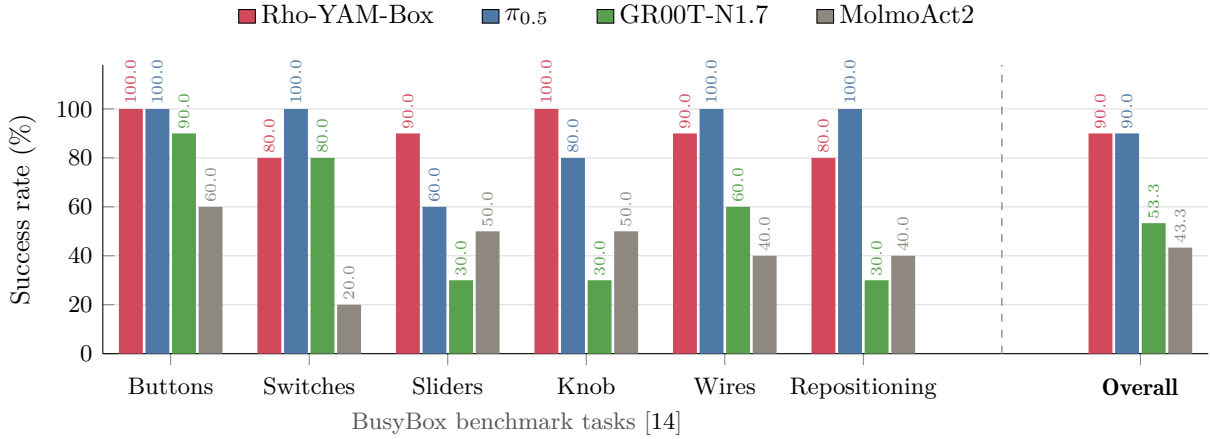


Figure 9 YAM Box results. Rho-YAM-Box outperforms GR00T-N1.7 and MolmoAct2 and ties with $\pi_{0.5}$, which had thousands of hours of data from robots similar to YAM Box in its posttraining mixture.

different initial configuration of the BusyBox, which determines the initial states of all of the BusyBox controls and the position and rotation of the BusyBox with respect to the robot.

Despite a lot of variability in the initial states of the BusyBox tasks as well as the precision and dexterity of manipulation required to complete them, each task generally takes an experienced teleoperator less than 15 seconds to complete. During evaluation, we set the task horizon to 900 steps, corresponding to 30 seconds on a YAM Box operating at 30 Hz. Thus, the tasks are fairly short-horizon, and the 2000 high-quality demonstrations across all tasks constitute a sufficiently large dataset to learn the tasks well.

Results analysis. As shown in Figure 9, Rho-YAM-Box’s average performance across the BusyBox suite matches or exceeds that of the other baselines, $\pi_{0.5}$ being the only model equal to Rho-YAM-Box in this experiment. This result validates two important theses.

First, midtrained and task-adapted Rho’s performance scales well with the amount of finetuning data: Rho doesn’t exhibit pathological behaviors that would prevent its behavior from achieving high success rates when offline finetuning data is plentiful.

Second, Rho’s architecture and/or training recipe enables data-efficient learning. Note that, to a first approximation, MolmoAct2 and $\pi_{0.5}$ were specialized to YAM Box thanks to their horizontal post-training on large amounts of demonstration data from embodiments similar to YAM Box: MolmoAct2’s training used 700 hours and $\pi_{0.5}$ likely used several thousand. Yet, despite Rho’s YAM Box pretraining and midtraining mixtures sampling less YAM Box data, its mean success rate on BusyBox is at par with $\pi_{0.5}$ ’s and significantly higher than MolmoAct2’s.

7.5.2 UR AI Trainer

Tasks and finetuning protocol. In the experiments on UR AI Trainer, we evaluate Rho’s performance on adaptation to multi-stage tasks in low-data finetuning regime. We adapt Rho’s $\pi_{0.5}$, and GR00T-N1.7 to two target tasks, training a separate policy for each task. Rho’s adaptation begins from the UR AI Trainer-midtrained checkpoint, Rho-UR-AI-Trainer, which captures the platform’s observations, kinematics, and control conventions.

- **Toolbox packing** (~ 150 demonstrations). The workspace contains an open toolbox and a tray filled with materials and tools. The robot must coordinate both arms across 4 task stages: lifting the tray, fitting it into the toolbox without tilting it or spilling its contents, closing the lid, and engaging the latch. Precision is paramount: without careful alignment of the tray with the toolbox sides, the toolbox’s lid won’t close. For the purpose of calculating

task progress, completing each stage earns the score of 0.25. The execution horizon during evaluation was 1080 steps (60 seconds @ 18 Hz), but all models finished their rollouts earlier.

- **Small-electronics cleanup** (~ 160 demonstrations). Two small electronic components (roughly, 1.5 inches long and 0.5 inches wide) are arbitrarily positioned on a tabletop in the beside a compartmentalized tray. All of the tray’s compartments but 2 are filled with other small electronics. The robot must pick and place each component on the table into an empty compartment of the tray. Despite being a pick-and-place task it presents difficulty due to the electronics pieces having small grippable surface, comparable in size to UR AI Trainer’s gripper tips, and the tray compartments being small as well. As in *Toolbox packing*, each stage (picking up and placing each component into an empty compartment) yields 0.25 points toward task progress. The task horizon during execution was 1620 steps (90 seconds @ 18 Hz).

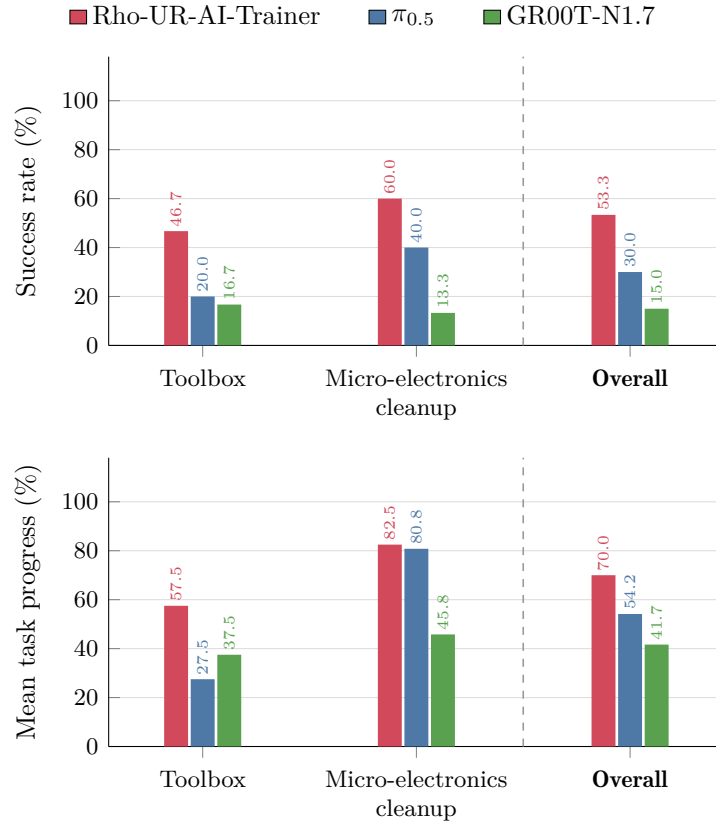


Figure 10 UR AI Trainer results. Rho-UR-AI-Trainer dominates the baselines in terms of success rate but is matched by $\pi_{0.5}$ on *Small-electronics cleanup*. This is caused by different failure modes of Rho and $\pi_{0.5}$. Rho sometimes fails to pick up one of the small electronics chips, capping its task progress at 0.5 in these cases, but precisely places picked-up electronics into the tray. $\pi_{0.5}$, on the other hand, tends to fail at placing one of the electronics pieces precisely, often putting it in an occupied compartment, missing one task stage.

Results analysis. The finetuned Rho-UR-AI-Trainer outperforms the other models by nearly $2\times$ on each task. This is likely due to the difficulty of the tasks themselves and $\pi_{0.5}$ ’s and GR00T-N1.7’s near-complete prior unfamiliarity with UR AI Trainer or its UR5e arms (see Appendix E.2): the data distributions of both baselines contain few demonstrations collected on dual-arm UR systems, if any, illustrating how Rho’s separation of adaptation to an embodiment and to a task makes the latter far more data-efficient.

At the same time, the long-horizon precision-demanding nature of both tasks makes them hard to learn from only 150-200 demonstrations even for Rho-UR-AI-Trainer, as indicated by all models’ absolute success rates. Task progress provides an additional perspective on relative performance. Although Rho-UR-AI-Trainer’s mean task progress is still higher, on *Small-electronics cleanup* it is nearly matched by $\pi_{0.5}$, whose failures tend to happen in the task’s concluding stages (placing electronics into the correct compartment).

7.5.3 FR3 Duo

Tasks and finetuning protocol. On FR3 Duo, we evaluate task adaptation across three task families selected to exercise complementary capabilities of the platform: long-horizon bimanual coordination, object–object alignment and stability, and precise small-object contact. Plug insertion contributes one evaluation variant, while tumbler and test-tube manipulation are each evaluated in both directions, yielding five variants from three finetuning datasets:

- **Plug insertion** (~ 150 demonstrations) requires one arm to stabilize a power strip while the other removes, reorients, and inserts the adapter into a second strip. This tests distinct bimanual roles over a long sequence ending in precise alignment. For measuring task progress, we break the task into 2 stages (picking up the plug and inserting it correctly) and assign 0.5 points for each.
- **Tumbler stacking and unstacking** use ~ 100 demonstrations, divided evenly between the two directions. The tall, easily disturbed objects test coordinated alignment and stability during stacking, as well as controlled separation and placement during unstacking. Both forward and backward task flavors consist of 4 stages (picking up each of 2 tumblers and placing it, either on the tabletop or on the tumbler below it), earning 0.25 points per stage.
- **Test-tube racking and unranking** use ~ 300 demonstrations, again divided evenly. These variants combine small-object handling and cork manipulation with constrained insertion into or extraction from a rack. These tasks, too, consist of 4 stages of picking up/placing the tube and the cork, inserting/removing the cork into/from the tube, and putting the closed tube into the rack/removing it from the rack, with 0.25 task progress points for each stage.

The paired tumbler and test-tube variants let us compare precision assembly with the generally less constrained reverse behavior while keeping the objects and workspace fixed. For each dataset, the finetuned Rho is initialized from the FR3 Duo-midtrained checkpoint Rho-FR3-Duo, while $\pi_{0.5}$ and GR00T-N1.7 begin from their respective pretrained checkpoints. We adapt all three models on the same demonstrations for 50k optimizer steps with a global batch size of 64.

For each task variant and model, we evaluate 30 rollouts from distinct initial configurations. The matched configuration set varies the positions and orientations of the adapter and power strips for plug insertion, the starting tumbler poses for stacking and unstacking, and the tube, cork, and rack poses for racking and unranking. Every model is evaluated on the same configurations.

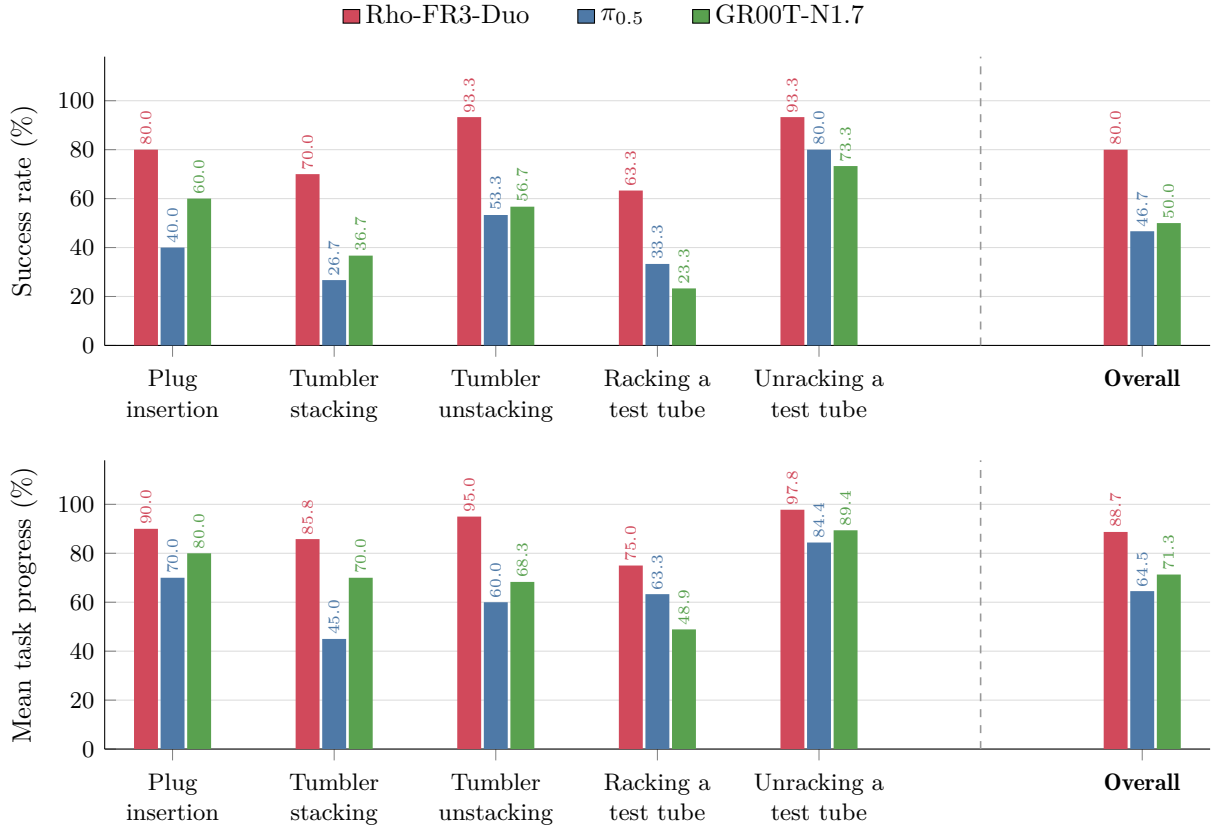


Figure 11 FR3 Duo task-adaptation results. Each model is adapted independently on each task dataset and evaluated over 30 distinct, matched initial configurations per variant. We report binary success rate and mean task progress; Overall is the unweighted mean across the five task variants.

Results analysis. As shown in Figure 11, Rho-FR3-Duo achieves 80.0% mean success and 88.7% mean task progress across the five variants, compared with 50.0% success and 71.3% progress for GR00T-N1.7 and 46.7% success and 64.5% progress for $\pi_{0.5}$. More importantly, the per-task success rates reflects the physical difficulty of the suite. Stacking is harder than unstacking and racking is harder than unracking for all 3 models: the forward task variants require the policy to establish and maintain precise alignment under contact, whereas the reverse tasks provide more room for error once the objects have been separated. Rho-FR3-Duo retains its largest advantages on these alignment-sensitive behaviors, leading the strongest baseline by 33.3 percentage points on *Tumbler Stacking* and by 30.0 points on test-tube racking. It also leads by 20.0 points on plug insertion, where success depends on coordinating distinct roles for the two arms over a long sequence.

Task progress adds useful context to the remaining failures. On *Tumbler Stacking*, for example, Rho-FR3-Duo reaches 85.8% mean progress despite 70.0% binary success; on test-tube racking it reaches 75.0% progress and 63.3% success. This pattern is consistent with many unsuccessful rollouts reaching the later alignment or placement stages rather than failing at the initial grasp. Rho-FR3-Duo leads both baselines in progress as well as success on every variant, suggesting that the gain is not limited to crossing the final success threshold.

Together with the controlled midtraining study in Section 7.3.2, these results illustrate the intended role of embodiment midtraining. A task-specific demonstration set necessarily covers only a narrow portion of the robot’s reachable workspace. A model adapted directly from a general pretrained checkpoint must therefore learn the task while also inferring the platform’s reachability and motion geometry from that limited slice of experience. Rho-FR3-Duo, in contrast, has already encountered a broader range of FR3 Duo poses and motions during embodiment midtraining.

This provides a stronger starting point when evaluation places objects near the edge of, or outside the configurations represented in, the task-finetuning data. The advantage is consistent with Rho-FR3-Duo’s gains across the pose-varied evaluation configurations, particularly on behaviors that require traversing the workspace before establishing precise contact.

Like the tasks from the UR AI Trainer experiments in Section 7.5.2, the FR3 Duo tasks are generally difficult to master from as few as 150 demonstrations collected offline from a fixed initial-state distribution. In the next section, we show how online adaptation using human feedback provided online for Rho’s task policies learned during offline finetuning can be a much more data-efficient way of achieving task proficiency than offline finetuning alone.

7.6 Online-adaptation results

The final experiments begin from offline task-adapted Rho policies and measure how much further they improve through online adaptation in latent space (Section 6.2). We enable Rho’s internal latent policy, a learned sampler that predicts the initial condition of its frozen flow-matching action expert and is stored in the same model checkpoint. This steers the generated action chunks without changing the pretrained model parameters [38, 56]. In all experiments below, the feedback signal is corrective supervision. On the physical robot a human operator takes over with a SpaceMouse; in simulation a scripted expert with access to privileged simulator state takes over. We assemble the actions actually executed during each query period into a complete chunk, traverse the discretized flow in reverse, and apply fixed-point refinement at each inverse step. Rho’s latent policy is trained to predict the resulting initial-condition targets [38]. Full algorithmic details and hyperparameters are given in Appendix F. We evaluate this stage in simulation on Meta-World [63] and on two FR3 Duo tasks.

Simulation. To obtain starting policies with substantial room for improvement, we train Rho with a deliberately limited budget on the full 50-task Meta-World suite and then adapt it in latent space on six challenging manipulation tasks that exercise precise insertion and extraction, tool use, and object placement, using 50 additional episodes per task supervised by Meta-World’s privileged scripted expert. Table 9 reports success over 100 evaluation rollouts per task. Latent-space adaptation raises average success from 57.5% to 73.3%. The largest gains are on Assembly (45% to 78%) and Stick Pull (58% to 78%), and on Hammer adaptation closes the remaining gap from 92% to 100%. Pick Out of Hole remains the hardest task, improving from 22% to 37%, while the gain on Basketball is small (+6 points).

Task	Offline	+ Online	Δ
Assembly	45.0	78.0	+33.0
Stick pull	58.0	78.0	+20.0
Pick out of hole	22.0	37.0	+15.0
Hand insert	73.0	86.0	+13.0
Hammer	92.0	100.0	+8.0
Basketball	55.0	61.0	+6.0
Average	57.5	73.3	+15.8

Table 9 Online latent-space adaptation on Meta-World. The offline policy is Rho trained with a limited budget on all 50 Meta-World tasks; the online column adapts it to each task in latent space from 50 episodes supervised by a privileged scripted expert. Success is measured over 100 rollouts per task; all values are percentages.

Physical robot. On FR3 Duo, we start from Rho-FR3-Duo policies adapted offline to the test-tube assembly and plug-insertion tasks from 150 demonstrations each, adapt them online from 15

additional human-supervised episodes, and evaluate each on a hard subset of initial configurations. Each subset consists of 10 configurations near the edge of the task-training distribution, with objects placed at the edge of the workspace or in unusual orientations; these are the same kinds of configurations on which FR3 Duo midtraining helped most in Section 7.3.2. The online corrections are collected on these configurations, so this experiment measures how quickly the policy can be adapted to a known weak region of its state distribution. Table 10 reports success rate (SR) and task progress (TP) over 30 trials (three per configuration). On test-tube assembly, online adaptation raises success from 9/30 to 21/30, more than doubling the success rate. On plug insertion, where the offline policy is already stronger, it raises success from 20/30 to 28/30, cutting failures from ten to two. In both cases, online supervision amounting to a tenth of the offline demonstration budget substantially improves the policy on exactly the configurations where it struggles most.

Task	Offline		+ Online	
	SR	TP	SR	TP
Test-tube assembly	30.0	56.7	70.0	81.7
Plug insertion	66.7	83.3	93.3	96.7

Table 10 Online latent-space adaptation on FR3 Duo. Offline policies are adapted from the Rho-FR3-Duo checkpoint on 150 demonstrations per task; online adaptation uses 15 additional human-supervised episodes. Each task is evaluated over 30 trials, three on each of 10 hard initial configurations. SR is success rate and TP is task progress; both are percentages.

8 Related work

8.1 Architectures of VLA models

VLA models [3, 13, 17, 19, 22, 35, 43, 49, 52, 60, 72] harness pretrained vision-language representations produced by VLM models for robot control, providing a route for transferring semantic and perceptual knowledge into manipulation policies. Early systems such as RT-2 established the VLA formulation [72] and generated actions autoregressively in a discretized space, while subsequent open models expanded the range of architectures, action representations, datasets, and deployment settings. Rho belongs to this family. It combines a 4.7B Phi-family backbone distilled from Phi-4-reasoning-vision-15B [2], a continuous 0.5B flow-matching action expert, and a training pipeline that separates multi-embodiment pretraining, embodiment-targeted midtraining, and task adaptation.

Architectures of VLAs with a continuous action expert (AE) differ mainly in *which* VLM representations condition the AE and *how* they are injected. OpenVLA-OFT [23] uses VLM’s final-layer features for AE conditioning. GR00T N1 [43] instead cross-attends to tokens from an intermediate LLM layer and discards the layers above it. SmolVLA [49] similarly keeps only the first half of the VLM but for the remaining VLM layers alternates cross- and self-attention in the AE. In π_0 and $\pi_{0.5}$ [3, 19], the AE is a transformer depth-matched to the VLM, and AE’s whose tokens attend to the VLM’s keys and values at every layer through block-causal shared attention. For some of these architectures, several works have studied mechanisms such as knowledge insulation (KI) for limiting interference between action learning and the VLM or for introducing action-aware intermediate representations [11, 15, 66]. Unfortunately, only few public studies have attempted to ablate these architectural decisions from the data recipe used for training the respective VLAs and from the amount of data used for pretraining. In one study [11] KI is shown to be beneficial, but this study was conducted on a $\pi_{0.5}$ -size dataset, likely over 10,000 hours of robot data. Fang et al. [13]’s study, done on MolmoAct2’s smaller-scale data, shows performance differences between layer-wise cross-attention-based architectures equipped

with KI and single-layer-conditioning without KI to be small. Since Rho’s data scale is closer to MolmoAct2’s and our experiments, too, showed little overall benefit of layer-wise cross-attention, we opted not to incorporate it into Rho’s architecture and instead optimize the shape of AE as described in Sections 3.2 and 7.2.

8.2 Other architectural paradigms in physical AI

A growing alternative to VLM-initialized VLAs builds robot policies on pretrained video generation models instead of vision-language models. This paradigm predates capable DiT-based video generation models such as Wan [57] and Cosmos [42]. PLEX [54] and UniPi [12] generated a text-conditioned video plan and recovered actions with a separately trained inverse dynamics model (IDM). GR-1 and GR-2 [5, 59] pretrained policies on large-scale video prediction before jointly finetuning on future frames and actions. The advent of Cosmos, Wan, and other strong pretrained video foundation models gave rise to two overlapping physical AI model classes. *Video-action models* (VAMs) [45], e.g., VPP [16], mimic-video [45], and GE-Act [31], treat the video backbone mainly as a source of predictive representations and attach a lightweight diffusion or flow-matching action decoder, which acts as an IDM, to intermediate video-model features, without necessarily rendering future frames. *World action models* (WAMs) [61], including DreamZero [61], Cosmos Policy [24], and UWM [71], instead emphasize joint generation: a single model predicts both future visual states and actions. LingBot-VA [29] and Fast-WAM [64] straddle the boundary between VAMs and WAMs, incorporating elements of both approaches.

Conceptually, VLAs differ from WAMs and VAMs mainly in the prior they inherit. VLAs get strong semantic and language grounding from static image-text pretraining but must learn physical dynamics from robot data alone. Video-based policies instead inherit spatiotemporal dynamics priors and receive dense per-frame supervision, but at the cost of substantially higher training cost and inference latency from long video-token sequences. Given only limited controlled comparisons between VAMs/WAMs and state-of-the-art VLAs [67] and opportunities to distill a physically-aware backbone from Phi, we chose to base Rho on a VLM. However, we envision future Rho-series models to video-generation backbones as this technology matures.

8.3 Physical grounding and VLA pretraining

The knowledge encoded by a general VLM is useful for robotics but is not necessarily optimized for spatial relationships, affordances, action consequences, or the observation distributions encountered by robots. Prior work improves this starting point through robotics-relevant vision-language supervision [13, 51] and mixes robot demonstrations with web-scale or embodied vision-language data during VLA training [13, 19, 32, 43, 72]. Rho similarly grounds its Phi-family backbone before action learning and retains physical-grounding data during multi-embodiment pretraining and embodiment midtraining. However, unlike most existing works such as Intelligence et al. [19], Rho performs the grounding by introducing the grounding data into the VLM’s training mixture (RT2), rather than simply post-training the VLM backbone on a combination of robot demonstrations and VQA data.

8.4 Data recipes, normalization and representations

The centerpiece of Rho’s data recipe is co-training, a concept that has been the mainstay of VLA development since at least RT-2 [72].

One set of key questions in co-training focuses on the kinds of data that should be included at each training stage. VLAs vary greatly in mixtures they use. Although some models are trained using robot demonstrations only [22], most VLAs rely on a combination of robot demonstrations with Web VQA [72] and even videos [20, 43] during pretraining, followed by robot-demonstrations data during later stages [3, 23]. Rho’s main difference from most existing approaches is the use of a multi-task multi-embodiment demonstration mixture that includes UMI-style data [9]

during pretraining, multi-task *single*-embodiment mixture during midtraining, and few-task single-embodiment during finetuning. An ablation study that converges on a similar recipe, with grounding VQA data mixed in during the first two stages, is Lin et al. [32]. This and other studies, e.g., Chen et al. [8] show that co-training on VQA data after pretraining is crucial for regularizing the VLA to prevent degradation of language understanding.

Another important data recipe consideration concerns the *quality* of co-training sub-datasets. Some works suggest that demonstrations of varying quality is acceptable and even desirable at pretraining in order to teach the model recovery behaviors [3], but data that standard for later training stages becomes significantly higher to make the learned behaviors fluent [19]. Yet other approaches observe that substandard data can be broadly helpful if it is annotated accordingly [20]. Rho uses a mix of public datasets [44] containing potentially suboptimal demonstrations and curated proprietary data for its target embodiments YAM Box, UR AI Trainer, and FR3 Duo, where erroneous trajectory segments are deliberately removed. During online task adaptation, it gets exposed to suboptimal demonstrations as well, but uses only their corrections for training.

Effectiveness of co-training on a multi-embodiment mixture of datasets also depends on the representations these datasets use for robots’ proprioception and actions. The π_0 [3] and $\pi_{0.5}$ [19] keeps each robot’s own action space, mostly joint space, and zero-pad state and action vectors to the dimensionality of the largest robot in the co-training mixture. RDT-1B [35] maps each robot’s actions into a shared 128-dimensional vector format, where every dimension has a fixed physical meaning, filling only the slots the robot has. GR00T N1 [43] uses a separate MLP per embodiment to project states and actions of different sizes into a shared embedding for the DiT. The aforementioned co-training study in Lin et al. [32] converts all data to EEf pose space. The EEf pose space, in turn, is sensitive to the choice of representation for EEf rotations. Rho is pretrained and midtrained in the EEf pose space; we use the 6D representation from Zhou et al. [70] and [53] due its continuity and chunk-start-relative actions, which mitigates memorization issues of absolute action spaces and avoids fast error accumulation typical of delta actions.

8.5 Embodiment and task adaptation

VLAs are commonly adapted to a target robot and task simultaneously through supervised fine-tuning [3, 13, 19, 43]. Rho explicitly separates these roles: embodiment midtraining uses a broad multi-task but single-robot dataset to familiarize the model with the robot platform’s possible behaviors and deployment environments, while downstream adaptation uses narrower demonstration sets to learn the target tasks. The controlled RoboTwin experiment (Section 7.3) and evaluations of Rho’s variants midtrained for YAM Box, UR AI Trainer, and FR3 Duo (Section 7.5) analyze the effectiveness of this separation.

Training stages conceptually related to Rho’s embodiment midtraining have been used in building several other VLAs [13, 19, 32] but with two notable differences.

First, while Rho uses midtraining as a means of *adapting* the pretrained model to a new embodiment, i.e., *generally* shifting the distribution of the model’s knowledge towards a particular robot and its use cases, $\pi_{0.5}$ [19] and MolmoAct2 [13] use it strictly as a tool for *specializing* the model, i.e. narrowing its capabilities. Namely, these works consider only the case where the target robot’s data is already present in the pretraining mixture, and the stage that follows pretraining focuses on the target robot data almost exclusively. Our experiment in Section 7.3 shows that embodiment training helps significantly after pretraining even if the model isn’t exposed to the target embodiment’s data during pretraining.

Second, MolmoAct2 [13] and $\pi_{0.5}$ [19] view embodiment-specific training as the final step in training pipeline and accordingly refer to it as *post-training* or *finetuning*, with the implicit hope that this stage will achieve zero-shot generalization on the target robot. Although zero-shot generalization on the target robot improves compared to models subjected only to multi-embodiment pretraining, in most scenarios it still falls far short of 90+% success rates. Rho’s

training recipe recognizes this limitation and focuses on using embodiment-targeted training as a midtraining step that prepares the model for data-efficient task-specific adaptation downstream.

Lin et al. [32] come closest to Rho’s use of embodiment training as a step followed by further model adaptation. Compared to their study, Rho’s evaluation in Section 7 shows that benefits of robot-targeted midtraining go beyond any single amount of finetuning data and hold across a wide range of data regimes (Figure 6) .

8.6 Online robot learning

Offline imitation policies are trained on states visited by their demonstrators, but deployment exposes them to states induced by their own errors. Online robot learning closes this distribution gap by collecting experience under the deployed policy and using it to improve the policy. Interactive imitation and reinforcement-learning systems have shown that human intervention can both keep collection safe and provide targeted learning signal at precisely these failure states [34, 36, 37].

Recent work extends this idea from task-specific controllers to generalist vision-language-action models. $\pi_{0.6}^*$ introduces RECAP, which combines demonstrations, autonomous rollouts, and teleoperated interventions, learns a value function from this heterogeneous experience, and improves the VLA through advantage-conditioned training [1]. This demonstrates that a large VLA can improve through on-robot practice rather than treating supervised post-training as its final stage. However, updating a full VLA online is computationally demanding and may alter capabilities inherited from pretraining.

Flow- and diffusion-based policies provide another adaptation surface: the latent initial condition from which the action-generation process begins. Latent-space reinforcement learning learns a policy over this variable from task rewards, while FlowDagger recovers latent targets by inverting corrected action chunks and trains on them with supervised regression [38, 56]. Both approaches can leave the pretrained action generator fixed, concentrating online updates in a substantially smaller module, although the adapted behavior remains constrained by the frozen generator. Rho incorporates this mechanism directly: its flow model can contain an internal latent policy that replaces random Gaussian sampling with an observation-conditioned initial condition and is saved as part of the same model checkpoint. Our experiments study the corrective-supervision variant, using scripted interventions in simulation and human interventions on the physical robot.

9 Limitations and future work

Rho is evaluated on a finite set of simulation benchmarks, robot embodiments, and manipulation tasks. These experiments do not establish universal generalization across hardware, environments, or task distributions. In particular, embodiment midtraining is studied on three physical bimanual platforms, and its benefits may depend on the diversity and quality of the embodiment-specific data. Our strongest-model claim is therefore restricted to the baselines, training budgets, and evaluation protocols reported in Section 7.

There are several important interconnected questions left open by our study. The first of them concerns Rho’s language steerability. We have not systematically evaluated how well Rho can follow diverse natural language instructions or how sensitive it is to variations in phrasing. This question remains underexplored for all other existing physical AI models for robotic manipulation that we are aware of; we hypothesize that this is due to all these models being designed for single-task finetuning, where practical importance of linguistic generalization has been limited. Nonetheless, zero-shot generalization in language, vision, and action space remains the North Star of physical AI, and the research community is seeing early signs of its advent [25]. We believe the importance of linguistic robustness of models like Rho and its descendants will become paramount in the near future.

The second question focuses on the effectiveness of Rho as part of agentic AI systems driven by frontier generative AI models and harnesses. Recent results indicate that performance of VLAs can be significantly enhanced when combined with GPT-6-Astra [50] acting as a governor, enabling more sophisticated reasoning, planning, and task execution capabilities in robotic manipulation scenarios. Exploring how well Rho performs in these workflows and, especially, how to make it stronger in this role by modifying Rho’s horizontal midtraining and vertical adaption recipe would help inform Rho’s practical deployment strategy. Answering the aforementioned question about the extent of Rho’s steerability in the language domain, the interaction interface between Rho and GPT-6-Astra-class models, is a crucial step in this direction.

10 Public release and responsible deployment

To support reproducibility and downstream use, we release the pretrained Rho model together with the embodiment-midtrained Rho-FR3-Duo, Rho-UR-AI-Trainer, and Rho-YAM-Box checkpoints. Alongside the model weights, we release code to fine-tune the checkpoints on downstream tasks, adapt them online using the latent-space methods studied in Section 6.2, and evaluate the released checkpoints. The release is intended to give practitioners a choice between adapting the general multi-embodiment checkpoint and starting from a checkpoint that has already learned the sensing and control conventions of a target platform. Last but not least, we release the dataset for BusyBox tasks collected on YAM Box: <https://huggingface.co/datasets/microsoft/BusyBox>

The released models are available on Microsoft Foundry and Hugging Face, and code for downstream fine-tuning and online adaptation is available on GitHub. .

When experimenting with Rho, we urge researchers and practitioners to follow responsible deployment practices. Offline task adaptation remains dependent on representative demonstrations, while online adaptation introduces additional operational and safety considerations. Interactive imitation learning requires timely and consistent corrections, and reinforcement learning depends on reward design and safe exploration. Online updates should be performed under human supervision with platform-specific safeguards, workspace constraints, and evaluation gates before deployment.

11 Conclusion

We presented Rho, an open-weights 5B-parameter vision-language-action model family for dual-arm robot manipulation built from physically grounded Phi-family backbone and a flow-matching action expert. In addition to the base Rho model, we release embodiment-midtrained checkpoints for I²RT YAM Box, Franka FR3 Duo, and Universal Robots AI Trainer – three dual-arm robot platforms that are emerging as a de-facto standard for bimanual robotic manipulation across research labs and industry.

The report studies the architectural choices that connect these components, the recipe used for multi-embodiment pretraining, and embodiment midtraining as an intermediate stage between general pretraining and task specialization. Across simulation and physical-robot experiments, we evaluate both the resulting base model and robot-specific checkpoints against strong open-weights baselines, establishing Rho as a strong contender to state-of-the-art VLAs across a broad range of tasks and embodiments. In particular, our experiments demonstrate the power of embodiment midtraining, which reduces the amount of offline finetuning data that needs to be collected on the target robot by half or, view differently, consistently improves task success rates after finetuning on a given small dataset by 10-20%. We further demonstrate that task-specialized Rho policies can continue adapting online in latent action space through interactive imitation learning and reinforcement learning. Together with the released pretrained and embodiment-midtrained checkpoints, these results provide a practical foundation for studying and deploying adaptable

robot policies.

12 Acknowledgments

We thank Microsoft Research Hardware Lab members Christopher O’Dowd, Teresa LaScala, Ibrahim Saeed, Mike Sheppard, and Lex Story for manufacturing robot parts and other experiment equipment; Vivan Amin and Ade Famoti for help in establishing collaborations with industrial partners; Ashley Llorens for sponsorship and guidance; Sriram Siva, K Wang, Xu Liu, Tim Chung, and Dan Rosenstein for sharing their experiences on dataset and model evaluations; and Abhishek Gupta and Patrick Yin from the University of Washington, Seattle for stimulating research conversations and insights on simulated data generation. We also thank Jyoti Aneja, Tyler LaBonte, and John Langford for their hard work on the Phi-4-reasoning-vision model that was used as the basis for this work.

We express our gratitude to Microsoft Research Early Access Program participant Teradyne Robotics for their technical support of our experiments on the UR AI Trainer robot, and thank Scale AI for collecting UR AI Trainer teleoperation data and iterating with us on the data collection protocol.

References

- [1] Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alex Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, and Zhiyuan Zhou. $\pi_{0.6}^*$: **a VLA that learns from experience**. *arXiv preprint arXiv:2511.14759*, 2025. URL <https://arxiv.org/abs/2511.14759>.
- [2] Jyoti Aneja, Michael Harrison, Neel Joshi, Tyler LaBonte, John Langford, and Eduardo Salinas. **Phi-4-reasoning-vision-15b technical report**. *arXiv preprint arXiv:2603.03975*, 2026. URL <https://arxiv.org/abs/2603.03975>.
- [3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : **A vision-language-action flow model for general robot control**. *arXiv preprint arXiv:2410.24164*, 2024. URL <https://arxiv.org/abs/2410.24164>.
- [4] Kevin Black, Manuel Y. Galliker, and Sergey Levine. **Real-time execution of action chunking flow policies**. In *NeurIPS*, 2025. URL <https://arxiv.org/abs/2506.07339>.
- [5] Chi-Lam Cheang, Guangzeng Chen, Ya Jing, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Hongtao Wu, Jiafeng Xu, Yichu Yang, Hanbo Zhang, and Minzhao Zhu. **Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation**. *arXiv preprint arXiv:2410.06158*, 2024. URL <https://arxiv.org/abs/2410.06158>.
- [6] Junsong Chen, Jincheng Yu, Chongjian Ge, Lewei Yao, Enze Xie, Yue Wu, Zhongdao Wang, James Kwok, Ping Luo, Huchuan Lu, and Zhenguo Li. **PixArt- α : Fast training**

- of diffusion transformer for photorealistic text-to-image synthesis. In *ICLR*, 2024. URL <https://arxiv.org/abs/2310.00426>.
- [7] Tianxing Chen, Zanzin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, Weiliang Deng, Yubin Guo, Tian Nian, Xuanbing Xie, Qiangyu Chen, Kailun Su, Tianling Xu, Guodong Liu, Mengkang Hu, Huan ang Gao, Kaixuan Wang, Zhixuan Liang, Yusen Qin, Xiaokang Yang, Ping Luo, and Yao Mu. **RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation**. In *ICML*, 2026. URL <https://arxiv.org/abs/2506.18088>.
 - [8] Xinyi Chen, Yilun Chen, Yanwei Fu, Ning Gao, Jiaya Jia, Weiyang Jin, Hao Li, Yao Mu, Jiangmiao Pang, Yu Qiao, Yang Tian, Bin Wang, Bolun Wang, Fangjing Wang, Hanqing Wang, Tai Wang, Ziqin Wang, Xueyuan Wei, Chao Wu, Shuai Yang, Jinhui Ye, Junqiu Yu, Jia Zeng, Jingjing Zhang, Jinyu Zhang, Shi Zhang, Feng Zheng, Bowen Zhou, and Yangkun Zhu. **InternVLA-M1: A spatially guided vision-language-action framework for generalist robot policy**. *arXiv preprint arXiv:2510.13778*, 2025. URL <https://arxiv.org/abs/2510.13778>.
 - [9] Cheng Chi, Zhenjia Xu, Chuer Pan, Eric Cousineau, Benjamin Burchfiel, Siyuan Feng, Russ Tedrake, and Shuran Song. **Universal manipulation interface: In-the-wild robot teaching without in-the-wild robots**. In *RSS*, 2024. URL <https://arxiv.org/abs/2402.10329>.
 - [10] Matt Deitke et al. **Molmo and pixmo: Open weights and open data for state-of-the-art vision-language models**. In *CVPR*, 2025.
 - [11] Danny Driess, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z. Ren, Homer Walke, Quan Vuong, Lucy Xiaoyang Shi, and Sergey Levine. **Knowledge insulating vision-language-action models: Train fast, run fast, generalize better**. In *NeurIPS*, 2025. URL <https://arxiv.org/abs/2505.23705>.
 - [12] Yilun Du, Mengjiao Yang, Bo Dai, Hanjun Dai, Ofir Nachum, Joshua B. Tenenbaum, Dale Schuurmans, and Pieter Abbeel. **Learning universal policies via text-guided video generation**. In *NeurIPS*, 2023. URL <https://arxiv.org/abs/2302.00111>.
 - [13] Haoquan Fang, Jiafei Duan, Donovan Clay, Sam Wang, Shuo Liu, Weikai Huang, Xiang Fan, Wei-Chuan Tsai, Shirui Chen, Yi Ru Wang, Shanli Xing, Jaemin Cho, Jae Sung Park, Ainaz Eftekhari, Peter Sushko, Karen Farley, Angad Wadhwa, Cole Harrison, Winson Han, Ying-Chun Lee, Eli VanderBilt, Rose Hendrix, Suveen Ellawela, Lucas Ngoo, Joyce Chai, Zhongzheng Ren, Ali Farhadi, Dieter Fox, and Ranjay Krishna. **MolmoAct2: Action reasoning models for real-world deployment**. *arXiv preprint arXiv:2605.02881*, 2026. URL <https://arxiv.org/abs/2605.02881>.
 - [14] Dean Fortier, Timothy Adamson, Tess Hellebrekers, Teresa LaScala, Kofi Ennin, Michael Murray, Andrey Kolobov, and Galen Mullins. **Benchmarking affordance generalization with BusyBox**. In *CoRL-2025 Eval & Deploy Workshop*, 2025. URL <https://arxiv.org/abs/2602.05441>.
 - [15] Ankit Goyal, Hugo Hadfield, Xuning Yang, Valts Blukis, and Fabio Ramos. **VLA-0: Building state-of-the-art vlans with zero modification**. *arXiv preprint arXiv:2510.13054*, 2025.
 - [16] Yucheng Hu, Yanjiang Guo, Pengchao Wang, Xiaoyu Chen, Yen-Jen Wang, Jianke Zhang, Koushil Sreenath, Chaochao Lu, and Jianyu Chen. **Video prediction policy: A generalist robot policy with predictive visual representations**. In *ICML*, 2025. URL <https://arxiv.org/abs/2412.14803>.

- [17] Hokyun Im, Euijin Jeong, Andrey Kolobov, Jianlong Fu, and Youngwoon Lee. **TwinVLA: Data-efficient bimanual manipulation with twin single-arm vision-language-action models**. In *ICLR*, 2026. URL <https://arxiv.org/abs/2511.05275>.
- [18] Physical Intelligence. **Open-world robotic manipulation with vision-language-action models**. 2024. URL <https://github.com/Physical-Intelligence/openpi>.
- [19] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. **$\pi_{0.5}$: a vision-language-action model with open-world generalization**. In *CoRL*, 2025. URL <https://arxiv.org/abs/2504.16054>.
- [20] Physical Intelligence, Bo Ai, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Greg Balke, Kevin Black, George Bokinsky, Shihao Cao, Thomas Charbonnier, Vedant Choudhary, Foster Collins, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Maitrayee Dhaka, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachlan Groom, Haroun Habeeb, Hunter Hancock, Karol Hausman, Gashon Hussein, Victor Hwang, Brian Ichter, Connor Jacobsen, Szymon Jakubczak, Rowan Jen, Tim Jones, Gregg Kammerer, Ben Katz, Liyiming Ke, Mairbek Khadikov, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Brendon LeCount, Sergey Levine, Xinyu Li, Adrian Li-Bell, Vladislav Lialin, Zhonglin Liang, Wallace Lim, Yao Lu, Enyu Luo, Vishnu Mano, Nandan Marwaha, Aikys Mongush, Liam Murphy, Suraj Nair, Tyler Patterson, Karl Pertsch, Allen Z. Ren, Gavin Schelske, Charvi Sharma, Baifeng Shi, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Jiaming Tang, Jimmy Tanner, Shalom Tekeste, Marcel Torne, Kyle Vedder, Quan Vuong, Anna Walling, Haohuan Wang, Jason Wang, XuDong Wang, Chris Whalen, Samuel Whitmore, Blake Williams, Charles Xu, Sukwon Yoo, Lili Yu, Wuming Zhang, Zhuoyang Zhang, and Ury Zhilinsky. **$\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities**. *arXiv preprint arXiv:2604.15483*, 2026. URL <https://arxiv.org/abs/2604.15483>.
- [21] Suchae Jeong, Jaehwi Song, Haeone Lee, Hanna Kim, Jian Kim, Dongjun Lee, Dong Kyu Shin, Changyeon Kim, Dongyoon Hahm, Woogyool Jin, et al. **Learning multi-view spatial reasoning from cross-view relations**. In *CVPR*, 2026.
- [22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. **OpenVLA: An open-source vision-language-action model**. In *CoRL*, 2024.
- [23] Moo Jin Kim, Chelsea Finn, and Percy Liang. **Fine-tuning vision-language-action models: Optimizing speed and success**. In *RSS*, 2025. URL <https://arxiv.org/abs/2502.19645>.
- [24] Moo Jin Kim, Yihuai Gao, Tsung-Yi Lin, Yen-Chen Lin, Yunhao Ge, Grace Lam, Percy Liang, Shuran Song, Ming-Yu Liu, Chelsea Finn, and Jinwei Gu. **Cosmos policy: Fine-tuning video models for visuomotor control and planning**. *arXiv preprint arXiv:2601.16163*, 2026. URL <https://arxiv.org/abs/2601.16163>.
- [25] Andrey Kolobov, Alex Bewley, Hamidreza Kasaei, Roberto Calandra, Johannes V. S. Busch, Moritz Reuss, Jiaxu Xing, and Jen Jen Chung. **The 8th Robot Learning Workshop: Is Physical AI Going Zero-Shot?** NeurIPS, 2026. URL <https://www.robot-learning.ml/2026/>.

- [26] Hadas Kress-Gazit, Kunimatsu Hashimoto, Naveen Kuppaswamy, Paarth Shah, Phoebe Horgan, Gordon Richardson, Siyuan Feng, and Benjamin Burchfiel. **Robot learning as an empirical science: Best practices for policy evaluation**. *arXiv preprint arXiv:2409.09491*, 2024. URL <https://arxiv.org/abs/2409.09491>.
- [27] Filippo Lazzati, Kyle Stachowicz, William Chen, Alberto Maria Metelli, Andrew Wagenmaker, and Sergey Levine. **Why does action chunking improve behavioral cloning performance in robotic control?** *arXiv preprint arXiv:2608.02547*, 2026. URL <https://arxiv.org/abs/2608.02547>.
- [28] Jason Lee, Jiafei Duan, Haoquan Fang, Yuquan Deng, Shuo Liu, Boyang Li, Bohan Fang, Jieyu Zhang, Yi Ru Wang, Sangho Lee, Winson Han, Wilbert Pumacay, Angelica Wu, Rose Hendrix, Karen Farley, Eli VanderBilt, Ali Farhadi, Dieter Fox, and Ranjay Krishna. **MolmoAct: Action reasoning models that can reason in space**. In *CoRL*, 2025. URL <https://arxiv.org/abs/2508.07917>.
- [29] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, Yujun Shen, and Yinghao Xu. **Causal world modeling for robot control**. In *RSS*, 2026.
- [30] You Li, Heyu Huang, Chi Chen, Kaiyu Huang, Chao Huang, Zonghao Guo, Zhiyuan Liu, Jinan Xu, Yuhua Li, Ruixuan Li, et al. **Migician: Revealing the magic of free-form multi-image grounding in multimodal large language models**. *arXiv preprint arXiv:2501.05767*, 2025.
- [31] Yue Liao, Pengfei Zhou, Siyuan Huang, Donglin Yang, Shengcong Chen, Yuxin Jiang, Yue Hu, Jingbin Cai, Si Liu, Jianlan Luo, Liliang Chen, Shuicheng Yan, Maoqing Yao, and Guanghui Ren. **Genie envisioner: A unified world foundation platform for robotic manipulation**. *arXiv preprint arXiv:2508.05635*, 2025.
- [32] Fanqi Lin, Kushal Arora, Jean Mercat, Haruki Nishimura, Paarth Shah, Chen Xu, Mengchao Zhang, Mark Zolotas, Maya Angeles, Owen Pfannenstiehl, Andrew Beaulieu, and Jose Barreiros. **A systematic study of data modalities and strategies for co-training large behavior models for robot manipulation**. *arXiv preprint arXiv:2602.01067*, 2026. URL <https://arxiv.org/abs/2602.01067>.
- [33] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. **LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning**. In *NeurIPS-2023 Track on Datasets and Benchmarks*, 2023.
- [34] Huihan Liu, Soroush Nasiriany, Lance Zhang, Zhiyao Bao, and Yuke Zhu. **Robot learning on the job: Human-in-the-loop autonomy and learning during deployment**. In *RSS*, 2023.
- [35] Songming Liu, Lingxuan Wu, Bangguo Li, Hengkai Tan, Huayu Chen, Zhengyi Wang, Ke Xu, Hang Su, and Jun Zhu. **Rdt-1b: a diffusion foundation model for bimanual manipulation**. In *ICLR*, 2024.
- [36] Jianlan Luo, Perry Dong, Yuexiang Zhai, Yi Ma, and Sergey Levine. **RLIF: Interactive imitation learning as reinforcement learning**. In *ICLR*, 2024.
- [37] Jianlan Luo, Charles Xu, Jeffrey Wu, and Sergey Levine. **Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning**. *arXiv preprint arXiv:2410.21845*, 2024. URL <https://arxiv.org/abs/2410.21845>.

- [38] Michael Murray, Daphne Chen, Simran Bagaria, Dean Fortier, Tess Hellebrekers, Galen Mullins, Harshavardhan Gajarla, Oier Mees, Maya Cakmak, and Andrey Kolobov. **FlowDagger: Human-in-the-loop adaptation of generative robot policies in latent space**. In *CoRL*, 2026. URL <https://arxiv.org/abs/2607.08877>.
- [39] NVIDIA. **Isaac gr00t n1.7: Open foundation model for generalized humanoid control**, 2025. URL <https://huggingface.co/nvidia/GR00T-N1.7-3B>.
- [40] NVIDIA. **NVIDIA Isaac Sim**. <https://developer.nvidia.com/isaac/sim>, 2025.
- [41] NVIDIA. **NVIDIA Isaac Teleop and GR00T 1.7 Open VLA Model Available in LeRobot**. Hugging Face, 2026. URL <https://huggingface.co/blog/nvidia/nvidia-isaac-teleop-and-gr00t17-in-lerobot>.
- [42] NVIDIA, Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, Daniel Dworakowski, Jiaojiao Fan, Michele Fenzi, Francesco Ferroni, Sanja Fidler, Dieter Fox, Songwei Ge, Yunhao Ge, Jinwei Gu, Siddharth Gururani, Ethan He, Jiahui Huang, Jacob Huffman, Pooya Jannaty, Jingyi Jin, Seung Wook Kim, Gergely Klár, Grace Lam, Shiyi Lan, Laura Leal-Taixe, Anqi Li, Zhaoshuo Li, Chen-Hsuan Lin, Tsung-Yi Lin, Huan Ling, Ming-Yu Liu, Xian Liu, Alice Luo, Qianli Ma, Hanzi Mao, Kaichun Mo, Arsalan Mousavian, Seungjun Nah, Sriharsha Niverty, David Page, Despoina Paschalidou, Zeeshan Patel, Lindsey Pavao, Morteza Ramezanali, Fitsum Reda, Xiaowei Ren, Vasanth Rao Naik Sabavat, Ed Schmerling, Stella Shi, Bartosz Stefaniak, Shitao Tang, Lyne Tchapmi, Przemek Tredak, Wei-Cheng Tseng, Jibin Varghese, Hao Wang, Haoxiang Wang, Heng Wang, Ting-Chun Wang, Fangyin Wei, Xinyue Wei, Jay Zhangjie Wu, Jiashu Xu, Wei Yang, Lin Yen-Chen, Xiaohui Zeng, Yu Zeng, Jing Zhang, Qinsheng Zhang, Yuxuan Zhang, Qingqing Zhao, and Artur Zolkowski. **Cosmos world foundation model platform for physical AI**. *arXiv preprint arXiv:2501.03575*, 2025. URL <https://arxiv.org/abs/2501.03575>.
- [43] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi "Jim" Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, Joel Jang, Zhenyu Jiang, Jan Kautz, Kaushil Kundalia, Lawrence Lao, Zhiqi Li, Zongyu Lin, Kevin Lin, Guilin Liu, Edith Llontop, Loic Magne, Ajay Mandlekar, Avnish Narayan, Soroush Nasiriany, Scott Reed, You Liang Tan, Guanzhi Wang, Zu Wang, Jing Wang, Qi Wang, Jiannan Xiang, Yuqi Xie, Yinzhen Xu, Zhenjia Xu, Seonghyeon Ye, Zhiding Yu, Ao Zhang, Hao Zhang, Yizhou Zhao, Ruijie Zheng, and Yuke Zhu. **GR00T N1: An open foundation model for generalist humanoid robots**. *arXiv preprint arXiv:2503.14734*, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [44] Open X-Embodiment Collaboration, Abby O'Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alex Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, Andrew Wang, Andrey Kolobov, Anikait Singh, Animesh Garg, Aniruddha Kembhavi, Annie Xie, Anthony Brohan, Antonin Raffin, Archit Sharma, Arefeh Yavary, Arhan Jain, Ashwin Balakrishna, Ayzaan Wahid, Ben Burgess-Limerick, Beomjoon Kim, Bernhard Schölkopf, Blake Wulfe, Brian Ichter, Cewu Lu, Charles Xu, Charlotte Le, Chelsea Finn, Chen Wang, Chenfeng Xu, Cheng Chi, Chenguang Huang, Christine Chan, Christopher Agia, Chuer Pan, Chuyuan Fu, Coline Devin, Danfei Xu, Daniel Morton, Danny Driess, Daphne Chen, Deepak Pathak, Dhruv Shah, Dieter Büchler, Dinesh Jayaraman, Dmitry Kalashnikov, Dorsa Sadigh, Edward Johns, Ethan Foster, Fangchen Liu, Federico Ceola, Fei Xia, Feiyu Zhao, Felipe Vieira Frujeri, Freek Stulp, Gaoyue Zhou, Gaurav S. Sukhatme, Gautam Salhotra, Ge Yan, Gilbert Feng, Giulio Schiavi, Glen Berseth,

Gregory Kahn, Guangwen Yang, Guanzhi Wang, Hao Su, Hao-Shu Fang, Haochen Shi, Henghui Bao, Heni Ben Amor, Henrik I Christensen, Hiroki Furuta, Homanga Bharadhwaj, Homer Walke, Hongjie Fang, Huy Ha, Igor Mordatch, Ilija Radosavovic, Isabel Leal, Jacky Liang, Jad Abou-Chakra, Jaehyung Kim, Jaimyn Drake, Jan Peters, Jan Schneider, Jasmine Hsu, Jay Vakil, Jeannette Bohg, Jeffrey Bingham, Jeffrey Wu, Jensen Gao, Jiaheng Hu, Jiajun Wu, Jialin Wu, Jiankai Sun, Jianlan Luo, Jiayuan Gu, Jie Tan, Jihoon Oh, Jimmy Wu, Jingpei Lu, Jingyun Yang, Jitendra Malik, João Silvério, Joey Hejna, Jonathan Boohar, Jonathan Tompson, Jonathan Yang, Jordi Salvador, Joseph J. Lim, Junhyek Han, Kaiyuan Wang, Kanishka Rao, Karl Pertsch, Karol Hausman, Keegan Go, Keerthana Gopalakrishnan, Ken Goldberg, Kendra Byrne, Kenneth Oslund, Kento Kawaharazuka, Kevin Black, Kevin Lin, Kevin Zhang, Kiana Ehsani, Kiran Lekkala, Kirsty Ellis, Krishan Rana, Krishnan Srinivasan, Kuan Fang, Kunal Pratap Singh, Kuo-Hao Zeng, Kyle Hatch, Kyle Hsu, Laurent Itti, Lawrence Yunliang Chen, Lerrel Pinto, Li Fei-Fei, Liam Tan, Linxi "Jim" Fan, Lionel Ott, Lisa Lee, Luca Weihs, Magnum Chen, Marion Lepert, Marius Memmel, Masayoshi Tomizuka, Masha Itkina, Mateo Guaman Castro, Max Spero, Maximilian Du, Michael Ahn, Michael C. Yip, Mingtong Zhang, Mingyu Ding, Minh Heo, Mohan Kumar Srirama, Mohit Sharma, Moo Jin Kim, Naoaki Kanazawa, Nicklas Hansen, Nicolas Heess, Nikhil J Joshi, Niko Suenderhauf, Ning Liu, Norman Di Palo, Nur Muhammad Mahi Shafiullah, Oier Mees, Oliver Kroemer, Osbert Bastani, Pannag R Sanketi, Patrick "Tree" Miller, Patrick Yin, Paul Wohlhart, Peng Xu, Peter David Fagan, Peter Mitrano, Pierre Sermanet, Pieter Abbeel, Priya Sundaesan, Qiuyu Chen, Quan Vuong, Rafael Rafailov, Ran Tian, Ria Doshi, Roberto Mart'in-Mart'in, Rohan Bajjal, Rosario Scalise, Rose Hendrix, Roy Lin, Runjia Qian, Ruohan Zhang, Russell Mendonca, Rutav Shah, Ryan Hoque, Ryan Julian, Samuel Bustamante, Sean Kirmani, Sergey Levine, Shan Lin, Sherry Moore, Shikhar Bahl, Shivin Dass, Shubham Sonawani, Shubham Tulsiani, Shuran Song, Sichun Xu, Siddhant Haldar, Siddharth Karamcheti, Simeon Adebola, Simon Guist, Soroush Nasiriany, Stefan Schaal, Stefan Welker, Stephen Tian, Subramanian Ramamoorthy, Sudeep Dasari, Suneel Belkhale, Sungjae Park, Suraj Nair, Suvir Mirchandani, Takayuki Osa, Tanmay Gupta, Tatsuya Harada, Tatsuya Matsushima, Ted Xiao, Thomas Kollar, Tianhe Yu, Tianli Ding, Todor Davchev, Tony Z. Zhao, Travis Armstrong, Trevor Darrell, Trinity Chung, Vidhi Jain, Vikash Kumar, Vincent Vanhoucke, Wei Zhan, Wenxuan Zhou, Wolfram Burgard, Xi Chen, Xiangyu Chen, Xiaolong Wang, Xinghao Zhu, Xinyang Geng, Xiyuan Liu, Xu Liangwei, Xuanlin Li, Yansong Pang, Yao Lu, Yecheng Jason Ma, Yejin Kim, Yevgen Chebotar, Yifan Zhou, Yifeng Zhu, Yilin Wu, Ying Xu, Yixuan Wang, Yonatan Bisk, Yongqiang Dou, Yoonyoung Cho, Youngwoon Lee, Yuchen Cui, Yue Cao, Yueh-Hua Wu, Yujin Tang, Yuke Zhu, Yunchu Zhang, Yunfan Jiang, Yunshuang Li, Yunzhu Li, Yusuke Iwasawa, Yutaka Matsuo, Zehan Ma, Zhuo Xu, Zichen Jeff Cui, Zichen Zhang, Zipeng Fu, and Zipeng Lin. **Open X-Embodiment: Robotic learning datasets and RT-X models.** In *ICRA*, 2024.

- [45] Jonas Pai, Liam Achenbach, Victoriano Montesinos, Benedek Forrai, Oier Mees, and Elvis Nava. **mimic-video: Video-action models for generalizable robot control beyond vlas.** In *RSS*, 2026.
- [46] William Peebles and Saining Xie. **Scalable diffusion models with transformers.** In *ICCV*, 2023. URL <https://arxiv.org/abs/2212.09748>.
- [47] Physical Intelligence. **OpenPI: LIBERO Benchmark.** Official OpenPI repository, 2026. URL <https://github.com/Physical-Intelligence/openpi/tree/main/examples/libero>.
- [48] Moritz Reuss, Hongyi Zhou, Marcel Rühle, Ömer Erdiñ Yağmurlu, Fabian Otto, and Rudolf Lioutikov. **Flower: Democratizing generalist robot policies with efficient vision-language-**

- action flow policies.** *arXiv preprint arXiv:2509.04996*, 2025. URL <https://arxiv.org/abs/2509.04996>.
- [49] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. **SmolVLA: A vision-language-action model for affordable and efficient robotics.** *arXiv preprint arXiv:2506.01844*, 2025. URL <https://arxiv.org/abs/2506.01844>.
 - [50] Jiayi Su, Yixin Zheng, Mi Yan, Li Yi, Zhizheng Zhang, and He Wang. **GPT 6 Astra as an embodied policy.** Technical report and code, 2026. URL <https://github.com/anonymous-report-421/eval-of-gpt-6-astra-as-policy>.
 - [51] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, Montserrat Gonzalez Arenas, Travis Armstrong, Ashwin Balakrishna, Robert Baruch, Maria Bauza, Michiel Blokzijl, Steven Bohez, Konstantinos Bousmalis, Anthony Brohan, Thomas Buschmann, Arunkumar Byravan, Serkan Cabi, Ken Caluwaerts, Federico Casarini, Oscar Chang, Jose Enrique Chen, Xi Chen, Hao-Tien Lewis Chiang, Krzysztof Choromanski, David D’Ambrosio, Sudeep Dasari, Todor Davchev, Coline Devin, Norman Di Palo, Tianli Ding, Adil Dostmohamed, Danny Driess, Yilun Du, Debidatta Dwibedi, Michael Elabd, Claudio Fantacci, Cody Fong, Erik Frey, Chuyuan Fu, Marissa Giustina, Keerthana Gopalakrishnan, Laura Graesser, Leonard Hasenclever, Nicolas Heess, Brandon Hernaez, Alexander Herzog, R. Alex Hofer, Jan Humplik, Atil Iscen, Mithun George Jacob, Deepali Jain, Ryan Julian, Dmitry Kalashnikov, M. Emre Karagozler, Stefani Karp, Chase Kew, Jerad Kirkland, Sean Kirmani, Yuheng Kuang, Thomas Lampe, Antoine Laurens, Isabel Leal, Alex X. Lee, Tsang-Wei Edward Lee, Jacky Liang, Yixin Lin, Sharath Maddineni, Anirudha Majumdar, Assaf Hurwitz Michaely, Robert Moreno, Michael Neunert, Francesco Nori, Carolina Parada, Emilio Parisotto, Peter Pastor, Acorn Pooley, Kanishka Rao, Krista Reymann, Dorsa Sadigh, Stefano Saliceti, Pannag Sanketi, Pierre Sermanet, Dhruv Shah, Mohit Sharma, Kathryn Shea, Charles Shu, Vikas Sindhwani, Sumeet Singh, Radu Soricut, Jost Tobias Springenberg, Rachel Sterneck, Razvan Surdulescu, Jie Tan, Jonathan Tompson, Vincent Vanhoucke, Jake Varley, Grace Vesom, Giulia Vezzani, Oriol Vinyals, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Fei Xia, Ted Xiao, Annie Xie, Jinyu Xie, Peng Xu, Sichun Xu, Ying Xu, Zhuo Xu, Yuxiang Yang, Rui Yao, Sergey Yaroshenko, Wenhao Yu, Wentao Yuan, Jingwei Zhang, Tingnan Zhang, Allan Zhou, and Yuxiang Zhou. **Gemini robotics: Bringing AI into the physical world.** *arXiv preprint arXiv:2503.20020*, 2025. URL <https://arxiv.org/abs/2503.20020>.
 - [52] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. **Octo: An open-source generalist robot policy.** In *RSS*, 2024. URL <https://arxiv.org/abs/2405.12213>.
 - [53] TRI LBM Team, Jose Barreiros, Andrew Beaulieu, Aditya Bhat, Rick Cory, Eric Cousineau, Hongkai Dai, Ching-Hsin Fang, Kunimatsu Hashimoto, Muhammad Zubair Irshad, Masha Itkina, Naveen Kuppaswamy, Kuan-Hui Lee, Katherine Liu, Dale McConachie, Ian McMahon, Haruki Nishimura, Calder Phillips-Graffin, Charles Richter, Paarth Shah, Krishnan Srinivasan, Blake Wulfe, Chen Xu, Mengchao Zhang, Alex Alspach, Maya Angeles, Kushal Arora, Vitor Campagnolo Guizilini, Alejandro Castro, Dian Chen, Ting-Sheng Chu, Sam Creasey, Sean Curtis, Richard Denitto, Emma Dixon, Eric Dusel, Matthew Ferreira, Aimee Goncalves, Grant Gould, Damrong Guoy, Swati Gupta, Xuchen Han, Kyle Hatch, Brendan Hathaway, Allison Henry, Hillel Hochshtein, Phoebe Horgan, Shun Iwase, Donovan Jackson, Siddharth

- Karamcheti, Sedrick Keh, Joseph Masterjohn, Jean Mercat, Patrick Miller, Paul Mitiguy, Tony Nguyen, Jeremy Nimmer, Yuki Noguchi, Reko Ong, Aykut Onol, Owen Pfannenstiehl, Richard Poyner, Leticia Priebe Mendes Rocha, Gordon Richardson, Christopher Rodriguez, Derick Seale, Michael Sherman, Mariah Smith-Jones, David Tago, Pavel Tokmakov, Matthew Tran, Basile Van Hoorick, Igor Vasiljevic, Sergey Zakharov, Mark Zolotas, Rares Ambrus, Kerri Fetzter-Borelli, Benjamin Burchfiel, Hadas Kress-Gazit, Siyuan Feng, Stacie Ford, and Russ Tedrake. **A careful examination of large behavior models for multitask dexterous manipulation.** *arXiv preprint arXiv:2507.05331*, 2025. URL <https://arxiv.org/abs/2507.05331>.
- [54] Garrett Thomas, Ching-An Cheng, Ricky Loynd, Felipe Vieira Frujeri, Vibhav Vineet, Mihai Jalobeanu, and Andrey Kolobov. **Plex: Making the most of the available data for robotic manipulation pretraining.** In *CoRL*, 2023.
- [55] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, Olivier Hénaff, Jeremiah Harmsen, Andreas Steiner, and Xiaohua Zhai. **SigLIP 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features.** *arXiv preprint arXiv:2502.14786*, 2025. URL <https://arxiv.org/abs/2502.14786>.
- [56] Andrew Wagenmaker, Yunchu Zhang, Mitsuhiro Nakamoto, Seohong Park, Waleed Yagoub, Anusha Nagabandi, Abhishek Gupta, and Sergey Levine. **Steering your diffusion policy with latent space reinforcement learning.** In *CoRL*, 2025. URL <https://proceedings.mlr.press/v305/wagenmaker25a.html>.
- [57] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wenten Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. **Wan: Open and advanced large-scale video generative models.** *arXiv preprint arXiv:2503.20314*, 2025. URL <https://arxiv.org/abs/2503.20314>.
- [58] Yi Ru Wang, Carter Ung, Christopher Tan, Grant Tannert, Jiafei Duan, Josephine Li, Anh Le, Rishabh Oswal, Markus Grotz, Wilbert Pumacay, Yuquan Deng, Ranjay Krishna, Dieter Fox, and Siddhartha Srinivasa. **RoboEval: Where robotic manipulation meets structured and scalable evaluation.** *arXiv preprint arXiv:2507.00435*, 2026. URL <https://arxiv.org/abs/2507.00435>.
- [59] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. **Unleashing large-scale video generative pre-training for visual robot manipulation.** *arXiv preprint arXiv:2312.13139*, 2023. URL <https://arxiv.org/abs/2312.13139>.
- [60] Wei Wu, Fan Lu, Yunnan Wang, Shuai Yang, Shi Liu, Fangjing Wang, Qian Zhu, He Sun, Yong Wang, Shuailei Ma, Yiyu Ren, Kejia Zhang, Hui Yu, Jingmei Zhao, Shuai Zhou, Zhenqi Qiu, Houlong Xiong, Ziyu Wang, Zechen Wang, Ran Cheng, Yong-Lu Li, Yongtao Huang, Xing Zhu, Yujun Shen, and Kecheng Zheng. **A Pragmatic VLA Foundation Model.** *arXiv preprint arXiv:2601.18692*, 2026. URL <https://arxiv.org/abs/2601.18692>.

- [61] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, Ayaan Malik, Kyungmin Lee, William Liang, Nadun Ranawaka, Jiasheng Gu, Yinzhen Xu, Guanzhi Wang, Fengyuan Hu, Avnish Narayan, Johan Bjorck, Jing Wang, Gwanghyun Kim, Dantong Niu, Ruijie Zheng, Yuqi Xie, Jimmy Wu, Qi Wang, Ryan Julian, Danfei Xu, Yilun Du, Yevgen Chebotar, Scott Reed, Jan Kautz, Yuke Zhu, Linxi "Jim" Fan, and Joel Jang. **World action models are zero-shot policies.** *arXiv preprint arXiv:2602.15922*, 2026. URL <https://arxiv.org/abs/2602.15922>.
- [62] Patrick Yin, Tyler Westenbroek, Zhengyu Zhang, Joshua Tran, Ignacio Dagnino, Eeshani Shilamkar, Numfor Mbiziwo-Tiapo, Simran Bagaria, Xinlei Liu, Galen Mullins, Andrey Kolobov, and Abhishek Gupta. **Emergent dexterity via diverse resets and large-scale reinforcement learning.** In *ICLR*, 2026. URL <https://arxiv.org/abs/2603.15789>.
- [63] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. **Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning.** In *CoRL*, 2020.
- [64] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. **Fast-wam: Do world action models need test-time future imagination?** *arXiv preprint arXiv:2603.16666*, 2026. URL <https://arxiv.org/abs/2603.16666>.
- [65] Wentao Yuan, Jiafei Duan, Valts Blukis, Wilbert Pumacay, Ranjay Krishna, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. **RoboPoint: A vision-language model for spatial affordance prediction for robotics.** In *CoRL*, 2025. URL <https://arxiv.org/abs/2406.10721>.
- [66] Lihan Zha, Asher J. Hancock, Mingtong Zhang, Tenny Yin, Yixuan Huang, Dhruv Shah, Allen Z. Ren, and Anirudha Majumdar. **LAP: Language-action pre-training enables zero-shot cross-embodiment transfer.** *arXiv preprint arXiv:2602.10556*, 2026. URL <https://arxiv.org/abs/2602.10556>.
- [67] Zhanguang Zhang, Zhiyuan Li, Behnam Rahmati, Rui Heng Yang, Yintao Ma, Amir Rasouli, Sajjad Pakdamansavoji, Yangzheng Wu, Lingfeng Zhang, Tongtong Cao, Feng Wen, Xinyu Wang, Xingyue Quan, and Yingxue Zhang. **Do world action models generalize better than vlas? a robustness study.** *arXiv preprint arXiv:2603.22078*, 2026. URL <https://arxiv.org/abs/2603.22078>.
- [68] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. **Learning fine-grained bimanual manipulation with low-cost hardware.** In *RSS*, 2023. URL <https://arxiv.org/abs/2304.13705>.
- [69] Enshen Zhou, Jingkun An, Cheng Chi, Yi Han, Shanyu Rong, Chi Zhang, Pengwei Wang, Zhongyuan Wang, Tiejun Huang, Lu Sheng, and Shanghang Zhang. **RoboRefer: Towards spatial referring with reasoning in vision-language models for robotics.** In *NeurIPS*, 2025. URL <https://arxiv.org/abs/2506.04308>.
- [70] Yi Zhou, Connelly Barnes, Jingwan Lu, Jimei Yang, and Hao Li. **On the continuity of rotation representations in neural networks.** In *CVPR*, 2019.
- [71] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. **Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets.** *arXiv preprint arXiv:2504.02792*, 2025. URL <https://arxiv.org/abs/2504.02792>.

- [72] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. **RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control**. In *CoRL*, 2023. URL <https://proceedings.mlr.press/v229/zitkovich23a.html>.

Appendix

A Model architecture details

This section records the complete model configuration used for Rho’s full multi-embodiment pretraining run. Rho couples the Phi-Phy backbone to a 12-block continuous action expert and conditions the expert on the backbone’s decoder-block-14 representation. Prediction and execution horizons are configurable properties of the training and deployment recipes; the widths, attention layout, and conditioning interface remain fixed.

Component	Configuration
Vision–language backbone	Phi-Phy; 4.68B parameters
Language decoder	32 causal-transformer blocks; width 3,072; MLP width 8,192; 32 query and 32 key/value heads
Text interface	Vocabulary size 100,352; configured context length 16,384
Vision encoder	SigLIP 2 SO400M NaFlex; 428M parameters; 16×16 patches; penultimate-layer visual features
Visual-token budget	256–3,600 tokens per image; robot images are resized with padding to 256×256 , yielding 256 visual patches per view
Cross-modal projector	Two-layer MLP, $1,152 \rightarrow 3,072 \rightarrow 3,072$, with GELU
Robot observation	One current observation, a language instruction, proprioceptive state, and up to three RGB camera views
Backbone output used by control	Joint image–language sequence after decoder block 14, before the remaining decoder blocks

Table 11 Phi-Phy backbone configuration used by Rho.

Component	Configuration
Action stream	One projected state token followed by one noisy-action token for every position in the action chunk
Transformer	12 blocks; width 2,048; feed-forward width 4,096; GELU activation; dropout 0.1 during pretraining
Block structure	Action-stream self-attention, cross-attention to the fixed VLM context, then a feed-forward update, each with a gated residual
Attention	16 attention heads, 4 key/value heads, head dimension 128; grouped-query attention is used in both attention modules
Normalization	DiT-style adaLN-Zero; shared adaLN-single base maps with zero-initialized block- and site-specific offsets
Position encoding	Fixed sinusoidal encoding; configured maximum sequence length 6,144
State/action width	State and action vectors are padded to 32 dimensions; masks exclude padded dimensions and invalid chunk positions from the loss
Flow input and output	Linear $32 \rightarrow 2,048$ noisy-action projection and $2,048 \rightarrow 32$ velocity head
Prediction horizon	50 positions during multi-embodiment pretraining; configurable during embodiment adaptation without changing transformer parameters

Table 12 Configuration of the Rho flow-matching action expert.

Signal	Source representation	Mapping	Use in action expert
Visual-language context	Block-14 Phi-Phy hidden sequence, width 3,072	Learned linear 3,072 $\rightarrow$ 2,048	Shared as the key/value sequence for cross-attention in every expert block
Robot state	One state vector padded to width 32	Learned linear 32 $\rightarrow$ 2,048	Prepended to the action-query sequence as one token
Noisy action	$H \times 32$ flow state	Linear action projection followed by a 4,096 $\rightarrow$ 2,048 $\rightarrow$ 2,048 action-time MLP	Supplies the remaining H query tokens
Flow time	Scalar t with a 2,048-dimensional sinusoidal embedding	2,048 $\rightarrow$ 2,048 $\rightarrow$ 2,048 SiLU MLP	Conditions the shared adaLN maps in every block
Padding masks	Valid image/text tokens, state/action dimensions, and chunk positions	No learned mapping	Masks attention or loss terms associated with padding

Table 13 The interface between Phi-Phy and the continuous action expert. The backbone context is computed once per observation and reused by all flow integration steps.

Component	Parameters	Pretraining status
Phi-Phy vision-language backbone	4.68B	Trainable except token embeddings
SigLIP 2 vision encoder	428M	Trainable; included above
Token-embedding table	308M	Frozen; included above
Action expert	542M	Trainable from random initialization
Complete Rho model	5.22B	Approximately 4.91B trainable

Table 14 Parameter accounting for Rho. Counts are rounded to the precision used throughout the paper.

Inference stage	Operation
Observation encoding	Encode all camera views and the instruction with Phi-Phy, extract decoder block 14, and project the valid context tokens to width 2,048. This backbone pass occurs once per policy query.
Latent initialization	Draw an independent Gaussian tensor $x_1 \in \mathbb{R}^{H \times 32}$; an internal learned latent policy can replace this draw during online adaptation (Appendix F).
Flow integration	Apply 10 uniform explicit-Euler steps from $t = 1$ to $t = 0$. Each step re-embeds the current action latent and queries the 12-block action expert while reusing the cached VLM context.
Action decoding	Project the final action tokens to 32 dimensions, remove padded dimensions, and invert the embodiment-specific normalization. No autoregressive text decoding is used to produce robot actions.
Receding-horizon execution	Execute a configurable prefix and then replan. Multi-embodiment pretraining uses $H = 50$ and a 25-action execution horizon; RoboEval uses 32/16 and LIBERO uses 16/8.
Numerics	Bfloat16 model execution with FlashAttention 2; the Euler state update is accumulated in float32 before conversion back to bfloat16.

Table 15 Rho inference path. Prediction and execution horizons are adapted to the control frequency and temporal extent of each embodiment.

B Training data and implementation details

B.1 Multi-embodiment pretraining

Component	Configuration
Initialization	Phi-Phy backbone; randomly initialized state/action projections and action expert
Trainable parameters	Vision encoder, cross-modal projector, language decoder, and action modules; token embeddings frozen
Robot inputs	One observation; up to three RGB views; padded 256×256 images; task instruction and proprioceptive state
Action targets	Chunk-relative EEf translation and 6D-rotation deltas; absolute grippers; approximately 1 s at the native rate; at most 50 steps; per-position 1st/99th-percentile normalization
Training objectives	Flow matching on robot batches; autoregressive cross-entropy on VQA and pointing batches, weighted by 0.02
Batch sampling	Robot/VL update probability 0.9/0.1; global batch 3,072/384
Optimizer	AdamW; $\beta_1 = 0.9$, $\beta_2 = 0.95$; $\epsilon = 10^{-6}$; weight decay 0.1
Peak learning rates	1×10^{-6} (vision-language modules); 1×10^{-4} (action modules)
Schedule	1,000-update linear warmup; stable phase; 45,000-update cosine decay to $0.1 \times$ peak
Regularization	Dropout 0.1; global gradient-norm clipping at 1.0
Numerics and compute	Bfloat16; 48 NVIDIA B200 GPUs

Table 16 Multi-embodiment pretraining recipe for Rho. Batch sizes count examples per modality-homogeneous optimization update.

B.2 Embodiment midtraining

For each target platform, embodiment midtraining continues from the multi-embodiment checkpoint using a broad, platform-specific robot dataset while retaining a small fraction of vision-language supervision. Table 17 gives the FR3 Duo recipe used to produce the checkpoint from which the FR3 task policies are initialized. Robot and vision-language batches are sampled in a 9:1 ratio. This stage preserves the pretraining action-expert architecture and optimizes the complete model, so the backbone, interface projections, and action expert can jointly acquire the target platform’s visual and control conventions.

Component	Configuration
Initialization	First-epoch multi-embodiment Rho checkpoint
Data mixture	FR3 Duo multi-task robot data and retained vision-language data, sampled at a 9:1 robot/VL ratio
Trainable parameters	Vision encoder, cross-modal projector, language decoder, and action modules; token embeddings frozen
Action representation	One-second chunk of chunk-relative Cartesian end-effector translation and 6D-rotation deltas with absolute gripper commands, padded to 50 positions
Batch and updates	128 examples per GPU on 8 B200 GPUs (global batch 1,024); 165,000 optimizer updates
Optimizer	AdamW; $\beta_1 = 0.9$, $\beta_2 = 0.95$; $\epsilon = 10^{-6}$; weight decay 0.1
Learning rates	1×10^{-6} for vision-language modules and 1×10^{-4} for the action modules
Schedule	1,000-update warmup, stable phase, then 15,000-update decay from 10^{-4} to 10^{-5}
Regularization and numerics	Dropout 0.1; global gradient-norm clipping at 1.0; bfloat16

Table 17 FR3 Duo embodiment-midtraining recipe.

B.3 Task finetuning

Task finetuning starts from the embodiment-midtrained checkpoint and uses only the demonstrations for the target task family. Table 18 summarizes the FR3 test-tube configuration, which is representative of the FR3 task-adaptation runs. Each model is adapted independently on the corresponding task demonstrations; the evaluation-specific data budgets are given in Section 7.5.

Component	Configuration
Initialization	FR3 Duo embodiment-midtrained Rho checkpoint
Data	Target-task demonstrations only; three RGB views, language, and proprioception
Trainable parameters	Vision encoder, cross-modal projector, language decoder, and action modules; token embeddings frozen
Action representation	One-second chunk of chunk-relative Cartesian end-effector translation and 6D-rotation deltas with absolute gripper commands, padded to a 50-position model chunk
Normalization	Per-task action-chunk mean and standard deviation
Batch and updates	16 examples per GPU on 4 B200 GPUs (global batch 64); 50,000 optimizer updates
Flow samples	One sample per training example
Optimizer	AdamW; $\beta_1 = 0.9$, $\beta_2 = 0.95$; $\epsilon = 10^{-6}$; weight decay 0.1
Learning rate	1×10^{-4} for all unfrozen parameters
Schedule	1,000-update warmup, stable phase, then 15,000-update decay from 10^{-4} to 10^{-5}
Regularization and numerics	Dropout 0.1; global gradient-norm clipping at 1.0; bfloat16

Table 18 Representative FR3 Duo task-finetuning recipe.

C Simulation evaluation protocols

Table 19 summarizes the LIBERO, RoboEval, and RoboTwin evaluation budgets. Unless stated otherwise, success is binary and a rollout is terminated as soon as the benchmark’s task predicate is satisfied or its time limit is reached. We average tasks uniformly, so tasks with longer horizons or more demonstrations do not receive greater weight.

Benchmark	Tasks	Adaptation	Trials / task / pass	Evaluation passes
LIBERO	40 (four suites)	40k updates	50	3
RoboEval	8	40k updates	100	3
RoboTwin	5 (Both Easy and Hard settings)	40k updates	100	3

Table 19 LIBERO, RoboEval, and RoboTwin task-adaptation and evaluation budgets for the reported Rho results. Each pass is a complete evaluation over the corresponding task set.

LIBERO. We use all 40 tasks in the four standard ten-task suites. For completeness, the task inventory is listed below; semicolon-separated entries denote distinct tasks.

Suite	Tasks
Spatial	Move the black bowl to the plate from: between the plate and ramekin; next to the ramekin; the table center; atop the cookie box; the cabinet’s top drawer; atop the ramekin; next to the cookie box; the stove; next to the plate; and atop the wooden cabinet.
Object	Place in the basket: alphabet soup; cream cheese; salad dressing; BBQ sauce; ketchup; tomato sauce; butter; milk; chocolate pudding; and orange juice.
Goal	Open the cabinet’s middle drawer; put the bowl on the stove; put the wine bottle atop the cabinet; open the top drawer and put the bowl inside; put the bowl atop the cabinet; push the plate in front of the stove; put the cream cheese in the bowl; turn on the stove; put the bowl on the plate; put the wine bottle on the rack.
Long	Put alphabet soup and tomato sauce in the basket; put cream cheese and butter in the basket; turn on the stove and put the moka pot on it; put the black bowl in the bottom drawer and close it; put the white mug on the left plate and the yellow-and-white mug on the right plate; put the book in the caddy’s rear compartment; put the white mug on the plate and the chocolate pudding to its right; put alphabet soup and cream cheese in the basket; put both moka pots on the stove; put the yellow-and-white mug in the microwave and close it.

Table 20 LIBERO task inventory used in our simulation evaluation. Spatial and Object hold the goal fixed while varying the relevant spatial relation or object; Long contains the benchmark’s ten multi-stage tasks.

We jointly adapt Rho on the official demonstrations from all four suites for 40k optimizer updates with global batch size 128 and eight flow samples per training example. At evaluation, each task is run for 50 trials using the benchmark-provided initial states. The horizon is 220 steps for Spatial, 280 for Object, 300 for Goal, and 520 for Long. A rollout succeeds when the official LIBERO task predicate is satisfied before its horizon. One pass therefore contains 500 trials per suite and 2,000 across all suites; the reported mean over evaluation seeds 42–44 contains 6,000 trials. Suite scores average their ten task success rates, and the overall score averages the four suite scores.

RoboEval. RoboEval comprises eight bimanual tasks: *Lift Pot* (grasp both handles and raise the pot), *Lift Tray* (grasp and lift the tray with both arms), *Pack Box* (close both box flaps), *Pick Single Book from Table*, *Rotate Valve* (rotate both valves counterclockwise), *Stack Single Book on Shelf*, *Stack Two Blocks*, and *Cube Handover* (transfer a cube between arms). We jointly adapt on all eight task datasets, sampling tasks uniformly. The reported Rho model uses bimanual 6D end-effector actions, global batch size 128, eight flow samples per training example, and the checkpoint after 40k optimizer updates. Action, state, and environment features use a single set of first- and 99th-percentile statistics pooled over all eight task datasets (89,765 frames); the same statistics are shared by every task during training and evaluation.

A RoboEval rollout succeeds when the environment’s binary task-success predicate is met within 250 environment steps. The policy uses an action-chunk size of 32 and a 16-action execution horizon. Each evaluation pass contains 100 trials per task (800 total), and we report the mean over three complete passes: 300 trials per task and 2,400 trials per model. Task progression and the behavioral measures in Table 7 and appendix G are computed from the same rollouts; they do not alter the binary success definition.

RoboTwin 2.0. The RoboTwin 2.0 study of midtraining effectiveness in Section 7.3.1 entails midtraining Rho for one of RoboTwin 2.0’s simulated robot embodiments using a multitask set of demonstrations provided by RoboTwin, finetuning the pretrained-only and midtrained Rho variants on RoboTwin tasks whose data is excluded from the midtraining mixture, and rolling out the resulting policies on these held-out tasks.

We choose the dual-arm UR5 robot with WSG grippers as our target RoboTwin 2.0 embodiment. RoboTwin provides demonstration data for 50 tasks. We use 45 tasks for midtraining and hold out 5 tasks for finetuning and evaluation: *Hanging Mug* (transfer a mug between arms and hang it on a rack), *Move Stapler to Pad*, *Place Burger and Fries on Tray*, *Microphone Handover*, and *Stack Three Bowls*.

For midtraining, we merge the demonstration data for 45 tasks, yielding roughly 23,000 demonstrations. We run midtraining for 2 epochs of 40K updates each, with a global batch size of 128 and 8 flow samples per training example.

For finetuning, the combined 5-task demonstration dataset contains 2,750 demonstrations. For measuring the effect of midtraining on finetuning datasets of different sizes, we carve out 12.5%, 25%, 50%, and 100% subsets of this combined dataset, where the 50% dataset contains the 25% dataset, and the 25% dataset contains the 12.5% dataset. We finetune the midtrained and pretrained-only Rho variants on each of these datasets, and compare their performance on 3 seeds.

Each of the reported Rho models uses bimanual 6D end-effector actions. Actions and states use a single set of mean and standard-deviation normalization statistics pooled over the 100% finetuning dataset and shared across all tasks during training and evaluation.

For each of the finetuning datasets and for each of the 3 seeds, we do one evaluation pass for each of the finetuned models. Each evaluation pass contains 50 trials per task, or 250 trials total; we measure the success rate of each pass and report the success rates averaged over 3 seeds. A RoboTwin 2.0 rollout succeeds when the environment’s native binary task-success predicate is met within 300 policy steps. Trials are matched across models: episode (i) for a given task uses scene and instruction seed (S+i), ensuring that checkpoints evaluated with the same base seed encounter identical scenarios. The policy predicts chunks of 32 actions and executes 16 actions before replanning. Each evaluation pass contains 50 trials per task, or 250 trials total.

RoboTwin 2.0 has two evaluation modes, Easy and Hard, and we report the results for each of them. In the Easy mode, the environment has the same fixed backgrounds, lighting, and table height as in the finetuning datasets, with no additional clutter. The Hard protocol randomizes backgrounds and lighting, adds table clutter, and varies table height by up to 3 cm. Hard-protocol results are averaged over base seeds 10000, 11000, and 13000 (these seeds yielded valid scenarios, unlike many others). Easy-protocol results use seeds 42-45.

D Physical-robot evaluation protocols

We use a common evaluation protocol across FR3 Duo, UR AI Trainer, and YAM Box, the parallel A/B evaluation [14, 26]. The evaluation proceeds in “blocks”, where every block consists in picking an initial state of the environment and sequentially running one trial for each model being compared starting from that initial state. Before each trial, an operator resets the robot and all manipulated objects to the initial state chosen for that block. Each initial state specifies the robot’s starting pose and the initial object arrangement. The evaluation is run for N initial states/blocks, and we report the success rate of each model across the N blocks. In essence, parallel A/B evaluation reduces the variance of physical-robot experiments by ensuring that each model is evaluated from the same set of initial conditions and that, for each initial condition, all models are evaluated on it at roughly the same time, minimizing environment drift.

Across initial states/blocks, we vary object position, orientation, and, where the task permits, object identity or arrangement. Camera poses, controller settings, language instructions, and the

physical workspace remain fixed within each comparison.

Each rollout begins after the reset is complete and runs autonomously until the task-specific time horizon, measured by the number of action execution steps, is reached or until an unrecoverable failure occurs. A trial is successful if and only if the robot completes every required task stage and all relevant objects are in their task-specified terminal configuration without human assistance when the task horizon is reached. This means that, e.g., an insertion must remain seated, a stacked object must remain stable, and articulated BusyBox controls must be at their target state at the time horizon, not just at some earlier point. Dropping a required object outside the reachable workspace, reaching an unrecoverable configuration, or triggering a safety stop counts as failure.

Human involvement during evaluation is limited to starting a trial, resetting the robot and scene between trials, and supervising safety; no teleoperation or corrective intervention is allowed during a scored rollout. An operator remains within reach of the emergency stop throughout execution. Low-level controllers enforce joint, reachability, action, and gripper limits, and communication watchdogs stop motion if commands are interrupted. The operator terminates a rollout if continued motion could damage the robot, task apparatus, or nearby objects. A trial interrupted by policy behavior is scored as a failure; a trial invalidated before meaningful policy execution by an unrelated camera, communication, or hardware fault is reset and repeated.

For multi-stage tasks, task progress is the fraction of predefined stages completed; it is reported as a diagnostic measure and does not change the binary success label.

The experiments on UR AI Trainer (Section 7.5.2) and FR3 Duo (Section 7.5.3) are done on multi-stage tasks. We finetune Rho-UR-AI-Trainer and Rho-FR3-Duo on each task’s data separately and use $N = 30$ evaluation blocks per task, i.e., each model’s success rate on each task in these experiments is computed over 30 rollouts. The YAM Box experiments (Section 7.5.1) use single-stage BusyBox tasks. We pool their YAM Box demonstration data, available at <https://huggingface.co/datasets/microsoft/BusyBox>, finetune a single policy for all BusyBox task classes from each base model, and carry out $N = 60$ evaluation blocks, 10 blocks for each task class. The overall scores on the right of Figures 9 10, and 11 weight task variants equally.

The online-adaptation evaluation uses a separate hard-initialization protocol, described in Section 7.6 and appendix F.

E Target embodiment descriptions

E.1 YAM Box

The YAM Box is a portable bimanual platform containing two six-degree-of-freedom I²RT YAM follower arms with motorized parallel-jaw grippers. We use three Intel RealSense D405 cameras: one fixed scene camera and one wrist camera per arm. The arms and cameras are calibrated into a common frame, and the two arm bases in our setup are separated by approximately 589 mm. Rho receives the three RGB views and the two end-effector poses and predicts bimanual Cartesian targets with position, 6D orientation, and absolute gripper commands. These targets are converted by the YAM control stack to joint-position commands and dispatched at 30 Hz. The workspace is the shared tabletop directly in front of the portable base; the adjustable arm spacing produces a central overlap in which either arm can reach the BusyBox and in which the box can be rotated or translated during the repositioning tasks.

E.2 UR AI Trainer

Our Universal Robots AI Trainer instance has two UR5e follower arms with Robotiq 2F-85 parallel-jaw grippers. The policy observes three Orbbec RGB streams: one fixed high scene view and one wrist-mounted view per arm. Cameras acquire at 60 Hz, and the latest synchronized images are consumed by the robot’s 18-Hz control loop. Both arms’ poses are transformed from their tilted base frames into a common world frame. The policy commands bimanual Cartesian

end-effector poses and absolute gripper positions in this frame; the deployment controller maps each target into the appropriate arm frame and tracks it with Cartesian impedance control. The physical workspace is the shared tabletop in front of the two inward-facing arms. Objects can be placed in either arm’s reachable region or in the central overlap used for coordinated lifting, packing, and handoff behaviors.

The physical midtraining data is augmented with successful trajectories from an OmniReset simulation of the UR5e and Robotiq 2F-85 in NVIDIA Isaac Sim. The simulation runs at 50 Hz with absolute Cartesian operational-space control and uses the same three-view policy interface as the physical platform. It contributes cube stacking, left- and right-arm placement, and left- and right-arm insertion tasks. We randomize object colors, camera poses, and focal lengths during collection, as described in Section 5.

E.3 FR3 Duo

The FR3 Duo platform comprises two seven-degree-of-freedom Franka Research 3 arms fitted with Robotiq parallel-jaw grippers. Visual observations come from one fixed ZED scene camera and one Intel RealSense camera mounted at each wrist. All three RGB streams are captured at 30 Hz. The two arms are expressed in a common world frame centered on the dual-arm mount. Rho predicts bimanual Cartesian end-effector targets—position, a continuous 6D orientation representation, and an absolute gripper command for each arm—at 30 Hz. At deployment, orientations are converted to quaternions and the pose targets are executed by the platform’s Cartesian impedance controllers; target poses that would violate joint limits are projected back to reachable poses. The task workspace is the shared tabletop region in front of the mount, including the overlap of the two arms’ reachable sets for bimanual handoffs, assembly, and object manipulation. Unimanual tasks use the corresponding arm’s portion of the same table and omit the unused wrist-camera stream.

F Online-adaptation details

Latent-space adaptation. Let $G_\theta(o, z)$ denote Rho’s flow solver, which maps an observation o and latent initial condition z to an action chunk a . Ordinarily, z is sampled from a Gaussian distribution. For online adaptation, we enable Rho’s optional internal latent policy $\pi_\phi(o) = z$, implemented as a learned noise-sampler submodule of the flow model. The resulting policy executes

$$a = G_\theta(o, \pi_\phi(o)). \quad (2)$$

We freeze all pretrained parameters θ and train only ϕ . Because the latent policy is a member of Rho’s flow model, it is trained in place, saved in the normal model state, and served as part of a single Rho checkpoint rather than as an external policy wrapper.

For an executed action chunk $a^\star$, we invert the frozen flow to recover a corresponding latent target,

$$z^\star \approx G_\theta^{-1}(o, a^\star). \quad (3)$$

The latent policy is then trained by supervised regression,

$$\mathcal{L}_{\text{latent}}(\phi) = \mathbb{E}_{(o, z^\star) \sim \mathcal{B}} \left[\|\pi_\phi(o) - z^\star\|_2^2 \right]. \quad (4)$$

Only the internal latent-policy parameters are updated; Rho’s vision-language backbone and flow-matching action expert remain frozen throughout online adaptation.

Noise inversion and refinement. We invert the same discretized flow used for action generation. Starting from the executed action chunk, we traverse the integration grid in reverse. Each inverse

Euler step is implicit because the velocity must be evaluated at the unknown state. We solve this equation using fixed-point iteration: an explicit reverse-Euler estimate initializes the step, after which the velocity field is repeatedly reevaluated at the current estimate. We use five fixed-point iterations per solver step in Meta-World and three on FR3 Duo. We invert the complete chunk executed during a query period, including both autonomous and supervisor-controlled portions when a takeover occurs. Because the flow need not be injective, this procedure recovers a preimage that reproduces the executed chunk rather than the unique noise originally sampled. We decode each recovered target through the forward flow and discard it if its round-trip mean-squared error exceeds 10^{-3} . Thus, refinement refers to the per-step fixed-point solution of the inverse trajectory; it does not update Rho’s pretrained parameters.

Internal latent policy. The latent policy reuses Rho’s own observation representation rather than introducing a separate visual encoder. It masked-mean-pools the vision-language prefix features, concatenates the current proprioceptive state, and applies a three-layer MLP with 1,024 hidden units per layer, layer normalization, and tanh activations. We calibrate the per-dimension mean and standard deviation of the pooled prefix features from the initial expert episodes before optimization. The MLP outputs a 16×32 latent initial condition, bounded by a final tanh to magnitude 3. We optimize the mean-squared latent-regression loss with Adam.

Correction interfaces. In Meta-World, a privileged scripted expert takes over at a randomly selected point in an episode and remains in control through its end. The policy is queried every eight environment steps and is adapted for 50 interaction episodes per task. On FR3 Duo, a human operator initiates a takeover when correction is needed and supplies end-effector corrections with a SpaceMouse. We collect 15 supervised episodes per task. In both cases, the actions actually executed over each query period are assembled into a complete chunk and inverted to produce the latent training target. Training and evaluation are run separately, with a fixed checkpoint used for evaluation rollouts.

Safety. The operator retains takeover and emergency-stop authority throughout physical data collection. The learned policy can change only the bounded latent initial condition supplied to the frozen action generator. Generated actions remain subject to the robot controller’s action, joint, and workspace limits and communication watchdogs. We save the adapted model through Rho’s normal checkpoint path after optimization and use a fixed checkpoint for evaluation.

Hyperparameter	Meta-World	FR3 Duo
Correction source	Scripted expert	Human (SpaceMouse)
Online episodes per task	50	15
Action/latent horizon	16	16
Latent dimension per step	32	32
Latent bound	3.0	3.0
Inversion method	Per-step fixed point	Per-step fixed point
Fixed-point iterations	5	3
Inversion-error threshold	10^{-3}	10^{-3}
Latent-policy input	VLM prefix + state	VLM prefix + state
Latent-policy MLP	$3 \times 1,024$	$3 \times 1,024$
Optimizer	Adam	Adam
Learning rate	10^{-4}	5×10^{-5}
Training batch size	256	256
Updates per optimization cycle	100	100

Table 21 Online latent-space adaptation hyperparameters. Rho’s vision-language backbone and flow-matching action expert are frozen in both settings; only its lightweight internal latent policy is optimized.

G RoboEval behavioral metrics

Table 22 gives the task-level breakdown of all RoboEval metrics, making task-specific performance differences visible.

Task	Model	Succ. $\uparrow$	TP $\uparrow$	BGVD $\downarrow$	CPL $\downarrow$	ECC $\downarrow$	JPL $\downarrow$	OPL $\downarrow$	SCC $\downarrow$	SC $\downarrow$
Cube handover	GR00T N1.7	0.64	0.93	0.043	1.35	0.16	6.81	4.14	0.373	0.477
	$\pi_{0.5}$	0.79	0.92	0.038	1.18	0.10	5.44	3.37	0.210	0.237
	MolmoAct2	0.86	0.95	0.036	1.11	0.04	5.12	3.30	0.107	0.193
	Rho	0.80	0.93	0.037	1.18	0.09	5.15	3.14	0.207	0.217
Lift pot	GR00T N1.7	0.67	0.81	0.058	2.57	0.13	16.83	9.70	0.120	0.603
	$\pi_{0.5}$	0.72	0.84	0.029	1.48	0.00	9.39	6.80	0.017	0.417
	MolmoAct2	0.57	0.77	0.026	1.61	0.00	9.32	6.83	0.003	0.570
	Rho	0.71	0.85	0.027	1.45	0.00	9.06	6.60	0.000	0.390
Lift tray	GR00T N1.7	0.96	0.99	0.027	0.85	0.00	5.84	4.62	0.083	0.083
	$\pi_{0.5}$	1.00	1.00	0.026	0.66	0.00	4.75	4.04	0.000	0.017
	MolmoAct2	0.96	0.99	0.029	0.71	0.00	4.81	3.94	0.007	0.057
	Rho	0.99	1.00	0.017	0.68	0.01	4.94	4.24	0.000	0.020
Pack box	GR00T N1.7	0.44	0.67	0.065	2.91	1.05	14.00	8.20	0.043	1.313
	$\pi_{0.5}$	0.53	0.74	0.064	3.23	0.92	14.99	9.17	0.067	1.080
	MolmoAct2	0.28	0.63	0.045	2.76	1.04	12.65	8.44	0.017	1.030
	Rho	0.50	0.75	0.055	2.99	1.34	13.90	8.77	0.057	1.367
Pick single book	GR00T N1.7	0.63	0.98	0.202	1.79	0.72	11.66	5.13	0.030	0.537
	$\pi_{0.5}$	0.72	0.99	0.197	1.40	0.63	8.80	4.14	0.000	0.613
	MolmoAct2	0.69	0.98	0.213	1.54	0.47	9.49	4.25	0.000	0.527
	Rho	0.85	1.00	0.214	1.33	0.46	8.44	3.84	0.000	0.480
Rotate valve	GR00T N1.7	0.59	0.84	0.015	0.45	6.79	3.48	2.55	0.000	0.000
	$\pi_{0.5}$	0.57	0.81	0.012	0.39	5.25	2.66	2.06	0.003	0.000
	MolmoAct2	0.60	0.86	0.014	0.41	5.83	2.98	2.28	0.000	0.000
	Rho	0.69	0.88	0.012	0.41	5.62	3.06	2.50	0.020	0.000
Stack single book shelf	GR00T N1.7	0.37	0.61	0.165	2.20	1.03	14.06	6.36	0.013	0.767
	$\pi_{0.5}$	0.61	0.73	0.178	2.12	0.86	13.28	6.14	0.000	0.923
	MolmoAct2	0.42	0.61	0.161	2.42	0.63	14.70	6.56	0.000	0.797
	Rho	0.77	0.85	0.171	2.05	0.56	12.73	5.78	0.000	0.767
Stack two blocks	GR00T N1.7	0.56	0.61	0.052	1.17	0.15	4.47	2.70	0.093	0.543
	$\pi_{0.5}$	0.43	0.53	0.049	1.17	0.18	4.55	2.83	0.003	0.567
	MolmoAct2	0.47	0.56	0.050	1.28	0.22	4.67	3.10	0.000	0.550
	Rho	0.55	0.59	0.054	1.16	0.06	4.28	2.63	0.000	0.577

Succ.: success rate; TP: task progression; BGVD: bimanual gripper vertical difference (m); CPL: Cartesian path length (m); ECC: environment collision count; JPL: joint path length (rad); OPL: orientation path length (rad); SCC: self-collision count; SC: slip count.

Table 22 Task-level RoboEval outcome and behavioral metrics after 40k task-adaptation steps. Each cell is pooled over three 100-episode evaluations for the corresponding task. Path-length measures sum the two arms. Arrows indicate the preferred direction.

H Authors’ contributions

Simran Bagaria designed and implemented synthetic data generation in NVIDIA Isaac Sim for midtraining Rho for UR AI Trainer. She contributed to maintaining the physical UR AI Trainer robot, collecting teleoperated task demonstrations on it, and finetuning Rho for various experiments.

Daphne Chen assisted with integrating online latent-space adaptation into Rho and running the online-adaptation experiments.

Dean Fortier made significant contributions to setting up and maintaining the project’s robot hardware, particularly FR3 Duo. He also established the initial teleoperation and data-collection pipeline, collected data for various experiments, and fine-tuned baseline models.

Jianlong Fu worked on WAM-based Rho variants, which are still undergoing evaluation at the time of writing.

Michael Harrison assisted with curation and evaluation of training datasets for Rho’s VLM backbone Phi-Phy relevant to Phi-Phy’s physical grounding.

Tess Hellebrekers designed and implemented early versions of Rho’s architecture, simulation experiments, and data processing pipelines. She set up data cleaning and conversion for pretraining. She experimented with several methods of incorporating tactile sensing into Rho that aren’t described in this report. She also set up and maintained UR AI Trainer, including teleoperation and tactile sensing hardware, its software control stack, and controller tuning.

Neel Joshi lead the development and evaluation of Rho’s VLM backbone Phi-Phy, including its architecture design, data curation, training, and evaluation.

Andrey Kolobov was the overall lead of Rho. He set the project’s goals and strategy, including Rho’s midtraining approach focusing on common dual-arm embodiments, oversaw its research and implementation across several teams, designed the experiments and the evaluation protocol, contributed to Rho’s architecture design, collected a significant fraction of the finetuning data for UR AI Trainer and YAM Box’s experiments, and carried out several of the experiments on them. He was also Rho’s data lead, defining Rho’s data recipe and mixtures for all training stages, procuring the proprietary data, and implementing and optimizing Rho’s data processing pipelines.

Dalton Moore made major contributions to setting up the software stack for YAM Box, collected much of teleoperation data on it and on UR AI Trainer, and finetuned Rho and baseline models on it.

Galen Mullins led the design, implementation, and optimization of Rho’s distributed training and evaluation infrastructure. He also assisted with Rho architecture ablations and made major contributions to Rho’s experiments by leading the setup of YAM Box’s software stack and evolving it, collecting some of teleoperation data for experiments on YAM Box, and implementing and running simulation experiments on RoboTwin 2.0 and LIBERO. He coordinated the work of several research engineers working on these aspects of the project and assisted Microsoft product teams in their evaluations of Rho.

Michael Murray led the design of Rho’s training recipe and architecture, including implementing shared AdaLN modulation, GQ attention, VL cotraining, knowledge insulation, and online latent-space adaptation; and running experiments determining their role in Rho, Rho’s final action-expert design, and pretraining and finetuning hyperparameters. He ran Rho pretraining and set up midtraining for FR3 Duo and UR AI Trainer. He also built the software stack for FR3 Duo, collected data for and ran offline finetuning on FR3 Duo, ran online-adaptation experiments on FR3 Duo and in Meta-World, and ran many of Rho’s LIBERO and RoboEval evaluations.

Eduardo Salinas contributed to engineering Phi-Phy’s training pipeline and inference.

Reuben Tan consulted the project team on the design of Rho training recipe and data augmentation strategies, and helped debug the Rho training pipeline.